Linux practise

Let me do both simultaneously — Git document first, then Docker Day 1!

Good. Now let me build the Git document first, then deliver Docker Day 1.

Git document ready! ✅ Now let's start Docker Day 1!

Docker Day 1 — Introduction & Core Concepts

Why Docker is Non-Negotiable for DevOps

Here is the most common problem in software development before Docker:

"It works on my machine."

Developer builds an app on their laptop — works perfectly. They deploy to server — crashes immediately. The server has a different version of Python, different libraries, different OS settings.

Docker solves this completely. You package your application with everything it needs — code, runtime, libraries, config — into one unit called a **container.** It runs identically everywhere.

Docker is in every DevOps job description. No exceptions.

What You Will Learn Today

- What Docker is and why it exists
- Containers vs Virtual Machines
- Docker architecture — images, containers, registry
- Installing Docker
- Your first container
- Core Docker commands
- Running real applications in containers
- Understanding Docker layers

What is Docker?

Docker is a platform that lets you **package, ship and run applications in containers.**

Think of a container like a shipping container on a cargo ship:

- Before shipping containers — every ship loaded cargo differently, goods got damaged, loading took days
- After shipping containers — one standard box, fits any ship, any truck, any crane — worldwide

Docker containers work the same way for software. One container format — runs on any laptop, any server, any cloud.

Containers vs Virtual Machines

This is one of the most asked interview questions:

```
Virtual Machine:                    Container:

┌──────────────────────┐            ┌──────────────────────┐
|  App A  |  App B     |            |  App A  |  App B     |
|─────────|────────────|            |─────────|────────────|
|Guest OS | Guest OS   |            |Libs A   | Libs B     |
|(full OS)| (full OS)  |            |──────────────────────|
|──────────────────────|            |  Docker Engine       |
|   Hypervisor         |            |──────────────────────|
|──────────────────────|            |  Host OS (one only)  |
|   Host OS            |            |──────────────────────|
|──────────────────────|            |  Hardware            |
|   Hardware           |            └──────────────────────┘
└──────────────────────┘

Size: GB each                      Size: MB each
Boot: minutes                      Boot: seconds
```

Feature	Virtual Machine	Container
Size	Several GB each	Tens of MB
Startup time	Minutes	Seconds
OS	Full OS per VM	Shares host OS kernel
Isolation	Complete	Process-level
Performance	More overhead	Near native
Use case	Full OS isolation	App packaging

Docker Architecture

```
Developer                    Docker Hub (Registry)
(your machine)               (cloud storage for images)

docker build  →  Image  →  docker push  →  Registry
                                ↑
                           docker pull
                                ↓
docker run    →  Container  ←  Image
```

Three key concepts:

Concept	What it is	Real world analogy
Image	Read-only template — blueprint	Recipe / class definition
Container	Running instance of an image	Dish made from recipe / object
Registry	Storage for images (Docker Hub)	App store for images

Installing Docker

```bash
# Ubuntu — official method:
sudo apt update
sudo apt install -y ca-certificates curl gnupg

# Add Docker GPG key:
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

# Add Docker repository:
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list

# Install Docker:
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io

# Verify installation:
docker --version
sudo docker run hello-world
```

Run Docker Without sudo

```bash
sudo usermod -aG docker $USER    # add yourself to docker group
newgrp docker                    # apply without logout
docker run hello-world           # now works without sudo
```

Your First Container

`docker run hello-world`

```bash
docker run hello-world
```

What happens step by step:

1. Docker looks for `hello-world` image locally — not found
2. Downloads it from Docker Hub automatically
3. Creates a container from that image
4. Runs it — prints a message
5. Container exits

You just ran your first container. The entire process took seconds.

Running a Real Container — nginx

```bash
docker run nginx
```

This downloads and starts nginx web server. But you cannot access it yet because ports are not mapped.

```bash
# Stop it first with Ctrl+C, then:
docker run -p 8080:80 nginx
```

Now go to `http://localhost:8080` in your browser — nginx is running in a container!

Flag	Meaning
`-p 8080:80`	Map port 8080 on your machine to port 80 in container

Flag	Meaning
`-d`	Detached mode — run in background
`-it`	Interactive terminal — stay inside container
`--name`	Give container a custom name

Running Detached — Background Mode

```bash
docker run -d -p 8080:80 --name my-nginx nginx
```

Container runs in background. You get your terminal back.

```bash
# Check it is running:
docker ps

# Access nginx:
curl http://localhost:8080
```

Core Docker Commands

Container Lifecycle Commands

```bash
# Run containers:
docker run nginx                        # run nginx
docker run -d nginx                     # run in background
docker run -it ubuntu bash              # run ubuntu and get shell inside
docker run -p 8080:80 nginx             # map ports
docker run --name my-app nginx          # custom name
docker run --rm nginx                   # auto-delete when stopped

# Managing running containers:
docker ps                               # list RUNNING containers
docker ps -a                            # list ALL containers including stopped
docker stop container_name             # stop gracefully
docker stop container_id               # stop by ID
docker start container_name            # start stopped container
docker restart container_name          # restart

# Removing containers:
```

```
docker rm container_name              # remove stopped container
docker rm -f container_name           # force remove running container
docker rm $(docker ps -aq)            # remove ALL stopped containers
```

Going Inside a Running Container

```bash
docker exec -it my-nginx bash          # open bash shell inside container
docker exec -it my-nginx sh            # if bash not available use sh
docker exec my-nginx ls /etc/nginx     # run single command in container
```

`-it` = interactive terminal. This is exactly like SSH-ing into the container.

Once inside you can look around, check configs, debug issues — just like a regular Linux system.

Viewing Container Logs

```bash
docker logs my-nginx                   # show all logs
docker logs my-nginx -f                # follow logs live — like tail -f
docker logs my-nginx --tail 50         # last 50 lines
docker logs my-nginx --since 10m       # logs from last 10 minutes
```

When your containerized application has an issue — logs are the first place to look.

Container Information

```bash
docker inspect my-nginx                # detailed JSON info — IP, mounts, config
docker stats                           # live CPU and memory usage of all container
docker stats my-nginx                  # stats for specific container
docker top my-nginx                    # processes running inside container
```

Image Commands

Working with Images

```bash
```

```bash
docker images                          # list all local images
docker images -a                       # list all including intermediate
docker pull nginx                      # download image without running
docker pull nginx:1.24                 # download specific version
docker pull ubuntu:22.04               # ubuntu specific version

docker rmi nginx                       # remove image
docker rmi nginx:1.24                  # remove specific version
docker rmi $(docker images -q)         # remove ALL images

docker image prune                     # remove unused images
docker image prune -a                  # remove ALL unused images
```

Image Tags — Versions

```bash
# Format: image_name:tag
nginx               # same as nginx:latest — always latest version
nginx:latest        # explicit latest
nginx:1.24          # specific version — USE THIS in production
ubuntu:22.04        # specific Ubuntu version
python:3.11-slim    # slim = smaller image
```

Always use specific versions in production. `latest` can change and break your application unexpectedly.

Searching for Images

```bash
docker search nginx                    # search Docker Hub
docker search --filter stars=100 nginx  # only popular images
```

Or visit hub.docker.com directly — better interface.

Understanding Docker Layers

What are Layers?

Every Docker image is built from layers — each instruction in a Dockerfile creates one layer. Layers are cached and shared between images.

```
nginx image layers:
Layer 4: nginx config files
Layer 3: nginx binary
Layer 2: Ubuntu apt packages
Layer 1: Ubuntu base OS
```

When you download a second image that also uses Ubuntu — Docker reuses the cached Ubuntu layer. Saves disk space and download time.

Seeing Layers

```bash
docker history nginx                    # see all layers of an image
docker history nginx --no-trunc        # see full commands
```

Docker System Commands — Housekeeping

Cleaning Up

```bash
docker system df                        # show disk usage by Docker
docker system prune                     # remove all stopped containers, unused imag
docker system prune -a                  # remove everything not currently running
docker system prune --volumes           # also remove unused volumes

# Individual cleanup:
docker container prune                  # remove all stopped containers
docker image prune                      # remove dangling images
docker volume prune                     # remove unused volumes
docker network prune                    # remove unused networks
```

Docker can use a lot of disk space over time. Run `docker system prune` regularly.

Real DevOps Scenario — Running a Full Application

Run a Database in Docker

```bash
```

```bash
# Run MySQL database in container:
docker run -d \
  --name mysql-db \
  -e MYSQL_ROOT_PASSWORD=secretpassword \
  -e MYSQL_DATABASE=myapp \
  -p 3306:3306 \
  mysql:8.0

# Verify it is running:
docker ps
docker logs mysql-db

# Connect to it:
docker exec -it mysql-db mysql -u root -p
```

You just ran a full MySQL database server in seconds. No installation. No configuration files. No dependencies. Just one command.

Run a Web Application

```bash
# Run a complete web app (example — nginx serving static files):
docker run -d \
  --name my-webapp \
  -p 8080:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:latest

# -v = volume mount — connects your local folder to container folder
# Changes you make to ./html appear immediately in the container
```

Quick Reference Diagram

```
docker pull image     →  Downloads image to local machine
docker run image      →  Creates container and starts it
docker ps             →  Shows running containers
docker stop name      →  Stops container
docker rm name        →  Removes container
docker images         →  Shows local images
docker rmi image      →  Removes image
docker logs name      →  View container output
docker exec -it bash  →  Shell inside container
```

Full Summary — Docker Day 1

Command	What it does
`docker run image`	Create and start container
`docker run -d image`	Run in background
`docker run -it image bash`	Interactive shell
`docker run -p 8080:80 image`	Map ports
`docker run --name name image`	Custom container name
`docker run --rm image`	Auto-delete on stop
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker stop name`	Stop container
`docker rm name`	Remove container
`docker logs name`	View logs
`docker logs -f name`	Follow logs live
`docker exec -it name bash`	Shell inside container
`docker images`	List local images
`docker pull image:tag`	Download image
`docker rmi image`	Remove image
`docker inspect name`	Detailed container info
`docker stats`	Live resource usage
`docker system prune`	Clean up unused resources

Interview Questions — Docker Day 1

Q1. What is Docker and why is it used? Answer: Docker is a containerization platform that packages applications with all their dependencies into portable containers. Used to solve the "works on my machine" problem — containers run identically on any system. Makes deployment consistent, fast and repeatable.

Q2. What is the difference between a container and a virtual machine? Answer: VMs run a full guest OS on a hypervisor — several GB each, minutes to start. Containers share the

host OS kernel — tens of MB, start in seconds. Containers are lighter and faster but provide process-level isolation. VMs provide complete OS isolation.

Q3. What is the difference between a Docker image and a container? Answer: An image is a read-only template — like a blueprint or class definition. A container is a running instance of an image — like an object created from a class. Multiple containers can run from the same image simultaneously.

Q4. What is Docker Hub? Answer: Docker Hub is a public registry — cloud storage for Docker images. Official images for nginx, mysql, ubuntu, python and thousands of others are hosted there. docker pull downloads from Docker Hub. docker push uploads your images there.

Q5. What does `docker run -d -p 8080:80 nginx` do? Answer: Runs nginx container in detached mode (background) and maps port 8080 on the host to port 80 inside the container. Accessing localhost:8080 on your machine reaches nginx running inside the container.

Q6. What are Docker layers? Answer: Docker images are built from stacked read-only layers — each Dockerfile instruction creates one layer. Layers are cached and shared between images. If two images use the same base OS layer, Docker stores it only once — saves disk space and speeds up downloads.

Q7. How do you view logs of a running container? Answer: `docker logs container_name` shows all logs. `docker logs -f container_name` follows logs live — like tail -f. `docker logs --tail 50 container_name` shows last 50 lines.

Q8. How do you get a shell inside a running container? Answer: `docker exec -it container_name bash` opens an interactive bash shell inside the running container. Use `sh` instead of `bash` if bash is not available (common in minimal images).

Homework — Before Docker Day 2

bash

```bash
# Practice all of these:
docker run hello-world
docker run -d -p 8080:80 --name my-nginx nginx
docker ps
curl http://localhost:8080
docker logs my-nginx
docker exec -it my-nginx bash
# Inside container: ls /etc/nginx/ then exit
docker stats my-nginx
docker stop my-nginx
docker rm my-nginx
docker images
docker system df
docker system prune
```

Your Progress

```
Linux  ━━━━━━━━━━━━━━━━━━━━━━━━  ✅ COMPLETE
Git    ━━━━━━━━━━━━━━━━━━━━━━━  ✅ COMPLETE
Docker ━┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅  Day 1 of 8 done
AWS    ┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅┅  (parallel chat)
```

Docker Day 2 we learn **Dockerfile — building your own custom images.** This is where Docker goes from running other people's software to packaging YOUR OWN application. Core skill for every DevOps engineer. 💪

Say **"Docker Day 2"** whenever you are ready!

Git github complete notes
Document · DOCX
Download

Docker Day 2 — Dockerfile & Building Custom Images

Why This is Critical for DevOps

Day 1 you ran other people's images — nginx, mysql, hello-world. That is useful but not enough.

In real DevOps work you need to:

- Package YOUR application into a Docker image
- Add your code, config and dependencies into the image
- Build it once — run it anywhere — dev, staging, production
- Push it to a registry so servers can pull and run it

The Dockerfile is how you do all of this. Every DevOps engineer writes Dockerfiles daily.

What You Will Learn Today

- What a Dockerfile is
- Every Dockerfile instruction explained
- Building images with `docker build`
- Build context and `.dockerignore`
- Layer caching — how to write efficient Dockerfiles

- Multi-stage builds — production best practice
- Real application Dockerfiles
- Common Dockerfile mistakes

What is a Dockerfile?

A Dockerfile is a **text file with instructions** that tells Docker how to build your image. Each instruction creates a layer.

```
Dockerfile instructions
        ↓
docker build
        ↓
Docker Image (your custom image)
        ↓
docker run
        ↓
Container (running application)
```

Your First Dockerfile

```bash
cd ~/linux_practice
mkdir docker_practice
cd docker_practice
```

Create a simple Dockerfile:

```bash
vim Dockerfile
```

```dockerfile
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y curl
CMD ["echo", "Hello from my first Docker image!"]
```

Build it:

```bash
docker build -t my-first-image .
```

Run it:

```bash
docker run my-first-image
# Output: Hello from my first Docker image!
```

You just built your first custom Docker image. Now let's understand every piece.

Every Dockerfile Instruction Explained

`FROM` — Base Image

Every Dockerfile must start with FROM. It sets the base image you build on top of.

```dockerfile
FROM ubuntu:22.04           # Ubuntu base
FROM debian:bookworm-slim   # Debian slim — smaller
FROM python:3.11-slim       # Python pre-installed
FROM node:18-alpine         # Node.js on Alpine — very small
FROM nginx:latest           # nginx web server
FROM scratch                # empty — for minimal Go/C binaries
```

Choosing the Right Base Image

```dockerfile
# Full images — large but complete:
FROM ubuntu:22.04           # ~77MB

# Slim images — reduced packages:
FROM python:3.11-slim       # ~45MB vs python:3.11 at ~350MB

# Alpine images — minimal Linux:
FROM node:18-alpine         # ~170MB vs node:18 at ~900MB

# Distroless — no shell, most secure:
FROM gcr.io/distroless/python3
```

`RUN` — Execute Commands During Build

`RUN` executes commands while building the image. Used to install packages, create directories, set permissions.

```dockerfile
```

```dockerfile
# Install packages:
RUN apt-get update && apt-get install -y nginx

# Multiple commands — WRONG way (creates multiple layers):
RUN apt-get update
RUN apt-get install -y nginx
RUN apt-get install -y curl

# Multiple commands — RIGHT way (one layer, one cache entry):
RUN apt-get update && \
    apt-get install -y \
        nginx \
        curl \
        wget \
    && rm -rf /var/lib/apt/lists/*
```

Why `rm -rf /var/lib/apt/lists/*`?

After installing packages, apt stores package lists in `/var/lib/apt/lists/`. These are only needed during installation — not at runtime. Deleting them reduces image size significantly.

```dockerfile
# Always clean up in the SAME RUN command:
RUN apt-get update && \
    apt-get install -y nginx && \
    rm -rf /var/lib/apt/lists/*

# WRONG — cleaning in separate RUN does NOT reduce image size:
RUN apt-get update && apt-get install -y nginx
RUN rm -rf /var/lib/apt/lists/*    # too late — layer already created with files
```

`COPY` — Copy Files Into Image

Copies files from your local machine into the image.

```dockerfile
COPY source destination

COPY app.py /app/app.py              # copy single file
COPY . /app/                         # copy everything from current directory
COPY config/ /etc/myapp/config/      # copy directory
COPY requirements.txt package.json /app/   # copy multiple files
```

`ADD` — Copy With Extra Powers

Similar to COPY but with extra features:

```dockerfile
ADD app.tar.gz /app/          # automatically extracts archives
ADD https://example.com/file /app/  # downloads from URL
```

Use COPY for simple file copying — it is more explicit and predictable. Use ADD only when you need auto-extraction.

`WORKDIR` — Set Working Directory

Sets the working directory for all following instructions. Creates the directory if it does not exist.

```dockerfile
WORKDIR /app
COPY . .        # copies to /app/
RUN npm install  # runs inside /app/
```

Always use WORKDIR instead of `RUN cd /app`. WORKDIR persists for the rest of the Dockerfile and for the running container.

```dockerfile
# WRONG:
RUN cd /app && npm install    # only affects this RUN command

# CORRECT:
WORKDIR /app
RUN npm install               # runs in /app and all subsequent commands too
```

`ENV` — Environment Variables

Sets environment variables inside the image. Available during build AND at runtime.

```dockerfile
ENV APP_PORT=8080
ENV DB_HOST=localhost
ENV ENVIRONMENT=production

# Use in Dockerfile:
EXPOSE $APP_PORT
```

```
# Available in running container:
# echo $APP_PORT → 8080
```

Override at runtime:

```bash
docker run -e APP_PORT=9090 myapp    # overrides the Dockerfile ENV
docker run --env-file .env myapp     # load from .env file
```

`EXPOSE` — Document Port

Documents which port the container listens on. Does NOT actually publish the port — it is informational.

```dockerfile
EXPOSE 80       # nginx uses port 80
EXPOSE 8080     # application uses 8080
EXPOSE 3306     # MySQL uses 3306
```

You still need `-p 8080:80` in `docker run` to actually map the port. EXPOSE just documents the intent and helps tools like docker-compose understand your container.

`CMD` — Default Command

Specifies the default command when container starts. Only the LAST CMD in the file is used.

```dockerfile
CMD ["nginx", "-g", "daemon off;"]    # exec form — preferred
CMD nginx -g "daemon off;"            # shell form — runs via /bin/sh -c
```

Exec form `["cmd", "arg"]` is preferred — does not start an extra shell process, handles signals properly.

Can be overridden at runtime:

```bash
docker run myimage                    # runs CMD
docker run myimage bash               # overrides CMD — opens bash instead
```

`ENTRYPOINT` — Fixed Command

Like CMD but cannot be overridden by normal `docker run` arguments. Arguments passed to `docker run` are APPENDED to ENTRYPOINT.

```dockerfile
ENTRYPOINT ["nginx"]
CMD ["-g", "daemon off;"]    # default arguments to ENTRYPOINT
```

```bash
docker run myimage                    # runs: nginx -g "daemon off;"
docker run myimage -t                 # runs: nginx -t (test config)
```

CMD vs ENTRYPOINT — The Key Difference

	CMD	ENTRYPOINT
Overridable	Yes — `docker run image other_cmd`	No — arguments appended
Use for	Default arguments	Fixed executable
Common use	Define defaults	Define the main program

```dockerfile
# Best practice — combine both:
ENTRYPOINT ["python3"]       # always runs python3
CMD ["app.py"]               # default file — overridable

# docker run myimage          → python3 app.py
# docker run myimage test.py  → python3 test.py
```

`ARG` — Build-Time Variables

Variables only available DURING the build — not in the running container.

```dockerfile
ARG APP_VERSION=1.0.0
ARG BUILD_DATE

RUN echo "Building version $APP_VERSION"
```

Pass at build time:

```bash
```

```
docker build --build-arg APP_VERSION=2.0.0 -t myapp .
```

`VOLUME` — Declare Mount Points

Declares a directory as a mount point for persistent data.

```dockerfile
VOLUME /var/lib/mysql          # database data persists here
VOLUME /var/log/nginx          # logs persist here
VOLUME ["/app/data", "/app/logs"]  # multiple volumes
```

`USER` — Run As Non-Root

Specifies which user to run the container as. Critical for security.

```dockerfile
# Create user and switch to it:
RUN useradd -r -s /bin/false appuser
USER appuser

# All following commands run as appuser not root
```

Never run production containers as root. If a container is compromised, running as root gives the attacker root access to the host system.

`HEALTHCHECK` — Monitor Container Health

Tells Docker how to check if the container is healthy.

```dockerfile
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
    CMD curl -f http://localhost:8080/health || exit 1
```

Flag	Meaning
`--interval`	How often to check
`--timeout`	How long to wait for response
`--retries`	How many failures before unhealthy

```bash
docker ps    # STATUS column shows: healthy / unhealthy / starting
```

`LABEL` — Metadata

Adds metadata to image — author, version, description.

```dockerfile
LABEL maintainer="varun@company.com"
LABEL version="1.0.0"
LABEL description="Production nginx server"
```

```bash
docker inspect myimage | grep Labels    # view labels
```

Building Images

`docker build` Command

```bash
docker build -t myapp:1.0 .
```

Part	Meaning
`-t myapp:1.0`	Tag — name and version of image
`.`	Build context — current directory

Useful Build Flags

```bash
docker build -t myapp .                     # basic build
docker build -t myapp:1.0 .                 # with version tag
docker build -t myapp:1.0 -t myapp:latest . # multiple tags
docker build -f Dockerfile.prod -t myapp .  # use specific Dockerfile
docker build --no-cache -t myapp .          # force rebuild — ignore cache
docker build --build-arg VERSION=2.0 -t myapp .  # pass build argument
```

What is Build Context?

The `.` in `docker build -t myapp .` is the build context — the directory Docker sends to the Docker daemon to build from. Every file in that directory is sent.

```bash
# If your project has large files:
du -sh .        # check total size

# Large build contexts slow down builds
# Use .dockerignore to exclude unnecessary files
```

`.dockerignore` — Exclude From Build Context

Works exactly like `.gitignore` but for Docker builds.

```bash
vim .dockerignore
```

```
# Dependencies — reinstalled during build anyway
node_modules/
vendor/
__pycache__/

# Git history — not needed in image
.git/
.gitignore

# Test files — not needed in production
tests/
*.test.js
*.spec.py

# Logs — not needed
*.log
logs/

# Secrets — NEVER in image
.env
*.pem
*.key

# Docker files themselves
Dockerfile
docker-compose.yml
.dockerignore
```

```
# OS files
.DS_Store
Thumbs.db
```

Layer Caching — Write Efficient Dockerfiles

How Caching Works

Docker caches each layer. If a layer's instruction has not changed — Docker uses the cache instead of rebuilding.

```dockerfile
FROM ubuntu:22.04          # cached ✅
RUN apt-get update         # cached ✅
RUN apt-get install nginx  # cached ✅
COPY . /app/               # CHANGED — cache invalidated here
RUN npm install            # must rebuild — runs again
```

When any layer changes — all subsequent layers rebuild from scratch.

The Golden Rule of Dockerfile Order

Put things that change LEAST at the top. Put things that change MOST at the bottom.

```dockerfile
# WRONG order — slow builds:
FROM node:18-alpine
COPY . /app/                  # code changes every time
WORKDIR /app
RUN npm install               # reinstalls ALL deps every build

# CORRECT order — fast builds with caching:
FROM node:18-alpine
WORKDIR /app
COPY package.json package-lock.json ./   # only changes when deps change
RUN npm install                          # cached unless deps change
COPY . .                                 # code changes often — at bottom
```

With correct ordering — if only your code changes, `npm install` uses cache. Only the final `COPY` layer rebuilds. Build is much faster.

Visualizing Layer Cache

```
Build 1 (first time — all layers build):
FROM node:18-alpine         → BUILD (download)
WORKDIR /app                → BUILD
COPY package.json ./        → BUILD
RUN npm install             → BUILD (downloads 200 packages)
COPY . .                    → BUILD
CMD ["node", "app.js"]      → BUILD

Build 2 (only code changed):
FROM node:18-alpine         → CACHED ✅
WORKDIR /app                → CACHED ✅
COPY package.json ./        → CACHED ✅ (package.json unchanged)
RUN npm install             → CACHED ✅ (200 packages NOT downloaded again)
COPY . .                    → REBUILD (code changed)
CMD ["node", "app.js"]      → REBUILD
```

Build 2 is seconds instead of minutes. This is why layer order matters enormously.

Multi-Stage Builds — Production Best Practice

The Problem With Single-Stage Builds

```dockerfile
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build           # compile/minify code
CMD ["node", "dist/app.js"]
```

This image includes:

- Node.js runtime
- npm and all build tools
- All development dependencies
- Source code AND compiled output
- Everything used only during build

Result: 900MB image in production. Slow to pull. Large attack surface.

Multi-Stage Build Solution

```dockerfile
# ── Stage 1: Build ──────────────────────────────
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# ── Stage 2: Production ─────────────────────────
FROM node:18-alpine AS production
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/package*.json ./
RUN npm install --production
USER node
EXPOSE 3000
CMD ["node", "dist/app.js"]
```

Result: 150MB instead of 900MB — build tools never enter the production image.

How Multi-Stage Works

```
Stage 1 (builder):           Stage 2 (production):
Full node:18 image           Tiny node:18-alpine
+ source code                + only compiled output
+ all dev dependencies       + only prod dependencies
+ build tools                + no build tools
         |                           |
         | COPY --from=builder       |
         └──────────────────────────>
                                     ↓
                            Final image — small and clean
```

Only the LAST stage becomes the final image. All earlier stages are discarded.

Multi-Stage for Python Application

```dockerfile
# ── Stage 1: Build dependencies ─────────────────
FROM python:3.11 AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user -r requirements.txt
```

```dockerfile
# ── Stage 2: Production ──────────────────────────
FROM python:3.11-slim AS production
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
USER nobody
EXPOSE 8080
CMD ["python3", "app.py"]
```

Real Dockerfile Examples

Example 1 — Static Website with Nginx

```dockerfile
FROM nginx:alpine

# Remove default nginx config
RUN rm /etc/nginx/conf.d/default.conf

# Copy custom nginx config
COPY nginx.conf /etc/nginx/conf.d/

# Copy website files
COPY html/ /usr/share/nginx/html/

EXPOSE 80

HEALTHCHECK --interval=30s --timeout=3s \
    CMD curl -f http://localhost/ || exit 1
```

Build and run:

```bash
docker build -t my-website .
docker run -d -p 8080:80 --name website my-website
curl http://localhost:8080
```

Example 2 — Python Flask Application

```dockerfile
```

```dockerfile
FROM python:3.11-slim

LABEL maintainer="varun@company.com"
LABEL version="1.0.0"

# Create non-root user
RUN useradd -r -s /bin/false flaskuser

WORKDIR /app

# Install dependencies first (caching optimization)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application
COPY . .

# Switch to non-root user
USER flaskuser

EXPOSE 5000

ENV FLASK_APP=app.py
ENV FLASK_ENV=production

HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD curl -f http://localhost:5000/health || exit 1

CMD ["python3", "-m", "flask", "run", "--host=0.0.0.0"]
```

Example 3 — Bash Script Runner (DevOps Tools Image)

dockerfile

```dockerfile
FROM ubuntu:22.04

LABEL maintainer="varun@company.com"
LABEL description="DevOps tools image"

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \
    apt-get install -y \
        curl \
        wget \
        git \
        vim \
```

```
        jq \
        unzip \
        python3 \
        python3-pip \
        awscli \
    && rm -rf /var/lib/apt/lists/*

# Install Terraform
ARG TERRAFORM_VERSION=1.5.0
RUN wget https://releases.hashicorp.com/terraform/${TERRAFORM_VERSION}/terraform_${T
    unzip terraform_${TERRAFORM_VERSION}_linux_amd64.zip && \
    mv terraform /usr/local/bin/ && \
    rm terraform_${TERRAFORM_VERSION}_linux_amd64.zip

WORKDIR /workspace

CMD ["/bin/bash"]
```

Build and use as interactive toolkit:

```bash
docker build -t devops-tools .
docker run -it --rm -v $(pwd):/workspace devops-tools
# Now you are inside container with all tools available
```

Common Dockerfile Mistakes

Mistake 1 — Running as Root

```dockerfile
# WRONG:
FROM ubuntu:22.04
RUN apt-get install -y myapp
CMD ["myapp"]          # runs as root — security risk

# CORRECT:
FROM ubuntu:22.04
RUN apt-get install -y myapp && \
    useradd -r appuser
USER appuser
CMD ["myapp"]
```

Mistake 2 — Secrets in Dockerfile

```dockerfile
# WRONG — password visible in image layers:
ENV DB_PASSWORD=supersecret123
RUN curl -u admin:supersecret123 https://api.example.com

# CORRECT — pass at runtime:
# docker run -e DB_PASSWORD=supersecret123 myapp
# or use Docker secrets in production
```

Mistake 3 — Not Using .dockerignore

```bash
# Without .dockerignore:
COPY . /app/          # copies node_modules (500MB), .git, .env, everything

# With .dockerignore excluding node_modules:
# Build context: 1MB instead of 500MB
# node_modules rebuilt from package.json — correct versions guaranteed
```

Mistake 4 — Using `latest` Tag

```dockerfile
# WRONG — unpredictable:
FROM node:latest

# CORRECT — pinned version:
FROM node:18.17.0-alpine3.18
```

Mistake 5 — Wrong Layer Order

```dockerfile
# WRONG — cache busted every time code changes:
COPY . /app/
RUN npm install

# CORRECT — cache npm install unless package.json changes:
COPY package*.json ./
```

```
RUN npm install
COPY . /app/
```

Full Summary — Docker Day 2

Instruction	What it does
`FROM image:tag`	Set base image — every Dockerfile starts here
`RUN command`	Execute command during build
`COPY src dst`	Copy files into image
`ADD src dst`	Copy with auto-extract and URL support
`WORKDIR /path`	Set working directory
`ENV KEY=value`	Set environment variable
`EXPOSE port`	Document which port app uses
`CMD ["cmd"]`	Default command — overridable
`ENTRYPOINT ["cmd"]`	Fixed command — args appended
`ARG name=default`	Build-time variable
`VOLUME /path`	Declare persistent mount point
`USER username`	Switch to non-root user
`HEALTHCHECK`	Define container health check
`LABEL key=value`	Add metadata

Command	What it does
`docker build -t name .`	Build image from Dockerfile
`docker build --no-cache -t name .`	Force full rebuild
`docker build -f Dockerfile.prod .`	Use specific Dockerfile
`docker history image`	See all layers

Interview Questions — Docker Day 2

Q1. What is a Dockerfile? Answer: A text file with instructions that tells Docker how to build a custom image. Each instruction creates a layer. Docker reads it top to bottom and builds the image by executing each instruction.

Q2. What is the difference between CMD and ENTRYPOINT? Answer: CMD sets the default command that can be overridden when running the container. ENTRYPOINT sets a fixed command that always runs — arguments from docker run are appended to it. Best practice is to combine both — ENTRYPOINT for the executable, CMD for default arguments.

Q3. What is the difference between COPY and ADD? Answer: COPY simply copies files from local machine to image. ADD does the same but also automatically extracts tar archives and supports downloading from URLs. Use COPY for simple file copying — it is more predictable. Use ADD only when you need auto-extraction.

Q4. Why should you combine RUN commands with && in Dockerfile? Answer: Each RUN instruction creates a new layer. Multiple RUN commands create multiple layers increasing image size. Combining with && creates one layer. Also clean up package caches in the SAME RUN command — cleaning in a separate RUN does not reduce size because the files already exist in the previous layer.

Q5. What is layer caching and why does instruction order matter? Answer: Docker caches each layer. When a layer changes all subsequent layers rebuild. Instructions that change least should be at the top — like installing dependencies. Instructions that change most go at the bottom — like copying source code. This way changing code does not invalidate the dependency cache.

Q6. What is a multi-stage build and why use it? Answer: A Dockerfile with multiple FROM statements where each stage builds on the previous. Used to separate build environment from production environment. Build tools, dev dependencies and source code stay in the build stage. Only compiled output and production dependencies go into the final stage. Results in much smaller and more secure production images.

Q7. Why should you never run containers as root? Answer: If a container running as root is compromised, the attacker gains root-level access. Create a dedicated non-root user with useradd in the Dockerfile and use USER instruction to switch to it before CMD. Principle of least privilege.

Q8. What is .dockerignore and why is it important? Answer: A file that tells Docker which files to exclude from the build context. Without it, large directories like node_modules or .git are sent to Docker daemon unnecessarily — slowing builds and potentially including secrets. Always exclude node_modules, .git, .env and test files.

Homework — Before Docker Day 3

```bash
cd ~/linux_practice/docker_practice
```

1. Write a Dockerfile for a simple Python script that prints "Hello DevOps"
2. Build it: `docker build -t hello-devops .`
3. Run it: `docker run hello-devops`
4. Create a `.dockerignore` file
5. Write a multi-stage Dockerfile — stage 1 installs something, stage 2 copies only what is needed
6. Run `docker history hello-devops` — see all layers
7. Try `docker build --no-cache -t hello-devops .` — watch it rebuild everything

Your Progress

```
Linux  ▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬  ✅ COMPLETE
Git    ▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬  ✅ COMPLETE
Docker ▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬  Day 2 of 8
AWS    ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  (parallel chat)
```

Docker Day 3 we learn **Docker Volumes and Networking** — how containers store data permanently and how they communicate with each other. This is what makes real multi-container applications work. 💪

Say **"Docker Day 3"** whenever you are ready!

Docker Day 3 — Volumes & Networking

Why This is Critical for DevOps

Containers have a fundamental problem — **they are stateless by default.**

When you stop and remove a container — everything inside it is gone. Database records, uploaded files, log files — all deleted. In production this is a disaster.

Another problem — containers need to talk to each other. Your web app container needs to reach your database container. Your nginx container needs to forward requests to your application container.

Volumes solve the data problem. Networking solves the communication problem. Both are essential for real applications.

What You Will Learn Today

- Why containers lose data without volumes

- Three types of storage in Docker
- Named volumes — the right way to persist data
- Bind mounts — connecting host files to containers
- Volume commands
- Docker networking basics
- Bridge, host and none networks
- Container-to-container communication
- Creating custom networks
- Real multi-container scenario

Part 1 — Docker Volumes

The Data Problem — Demo First

```bash
# Start a container and create a file:
docker run -it --name test-data ubuntu bash
# Inside container:
echo "important data" > /data/myfile.txt
exit

# Remove container:
docker rm test-data

# Run new container — file is GONE:
docker run -it --name test-data ubuntu bash
cat /data/myfile.txt    # No such file or directory
```

Every container starts fresh. No memory of previous run. For databases and file storage — this is unacceptable.

Three Types of Docker Storage

```
Host Machine
┌──────────────────────────────────────────┐
│                                          │
│  /var/lib/docker/volumes/  ←── Named Volume │
│  /home/varun/myapp/        ←── Bind Mount   │
│                                          │
│          ┌──────────────┐                │
│          │   Container  │                │
```

```
│          │  /app/data ──────── Named Volume │
│          │  /app/code ──────── Bind Mount   │
│          │  /tmp      ─────── tmpfs (memory) │
│          └──────────────┘                   │
└──────────────────────────────────────────────┘
```

Type	What it is	When to use
Named Volume	Docker-managed storage on host	Database data, persistent app data
Bind Mount	Map specific host folder to container	Development — live code reload
tmpfs	Memory only — lost on stop	Sensitive temp data, caching

Named Volumes — The Right Way

What is a Named Volume?

A named volume is storage that Docker manages for you. It lives in `/var/lib/docker/volumes/` on the host. Containers can attach to it — data persists even if the container is deleted.

```bash
# Create a named volume:
docker volume create mydata

# Run container with volume attached:
docker run -d \
  --name myapp \
  -v mydata:/app/data \
  ubuntu sleep infinity

# Write data inside container:
docker exec myapp bash -c "echo 'persistent data' > /app/data/file.txt"

# Remove container:
docker rm -f myapp

# New container — same volume — data is there:
docker run --rm \
  -v mydata:/app/data \
  ubuntu cat /app/data/file.txt
# Output: persistent data
```

Data survived the container deletion completely.

Volume Commands

```bash
docker volume create mydata           # create named volume
docker volume ls                      # list all volumes
docker volume inspect mydata          # detailed info — where it lives on host
docker volume rm mydata               # remove volume — deletes all data
docker volume prune                   # remove ALL unused volumes
```

Inspecting a Volume

```bash
docker volume inspect mydata
```

Output:

```json
[
    {
        "Name": "mydata",
        "Driver": "local",
        "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
        "Labels": {},
        "Scope": "local"
    }
]
```

The actual data lives at `/var/lib/docker/volumes/mydata/_data` on your host machine.

Real Example — MySQL with Persistent Data

```bash
# WITHOUT volume — data lost when container removed:
docker run -d --name mysql-bad mysql:8.0 -e MYSQL_ROOT_PASSWORD=pass

# WITH volume — data persists forever:
docker run -d \
  --name mysql-db \
  -e MYSQL_ROOT_PASSWORD=secretpass \
  -e MYSQL_DATABASE=myapp \
  -v mysql-data:/var/lib/mysql \
  -p 3306:3306 \
  mysql:8.0
```

```bash
# Even if you remove and recreate the container:
docker rm -f mysql-db
docker run -d \
  --name mysql-db \
  -e MYSQL_ROOT_PASSWORD=secretpass \
  -v mysql-data:/var/lib/mysql \
  -p 3306:3306 \
  mysql:8.0
# All your database records are still there!
```

Bind Mounts — Development Workflow

What is a Bind Mount?

A bind mount maps a **specific folder from your host machine** into the container. Changes on host appear immediately in container and vice versa.

```bash
# Syntax:
docker run -v /host/path:/container/path image

# Example:
docker run -d \
  --name dev-nginx \
  -p 8080:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx
```

Now:

- Edit `./html/index.html` on your laptop
- Refresh browser — changes appear immediately
- No rebuild needed

This is how developers work with Docker — live reload without rebuilding images.

Named Volume vs Bind Mount

```bash
# Named volume — Docker manages location:
-v mydata:/app/data                    # volume name : container path
```

```
# Bind mount — you control location:
-v /home/varun/myapp:/app/data        # host path : container path
-v $(pwd):/app                        # current directory
```

Named Volume	Bind Mount
Docker manages storage location	You specify exact host path
Works on any OS consistently	Path must exist on host
Best for production data	Best for development
Backed up with docker volume	Backed up normally with host
Cannot edit easily from host	Easy to edit from host

Read-Only Bind Mounts

```bash
# Container can read but NOT write:
docker run -v $(pwd)/config:/app/config:ro nginx

# Useful for:
# — Config files you don't want container to modify
# — Source code in production
# — SSL certificates
```

Volume in Dockerfile

```dockerfile
# Declares a mount point — Docker creates anonymous volume:
VOLUME /var/lib/mysql
VOLUME /app/data
```

When you run this image without specifying a volume — Docker automatically creates an anonymous volume for that path. Not recommended — use named volumes explicitly in `docker run` or `docker-compose`.

Backup and Restore Volumes

```bash
# Backup a volume to tar file:
docker run --rm \
```

```
  -v mysql-data:/data \
  -v $(pwd):/backup \
  ubuntu \
  tar -czf /backup/mysql-backup.tar.gz /data

# Restore volume from backup:
docker run --rm \
  -v mysql-data:/data \
  -v $(pwd):/backup \
  ubuntu \
  tar -xzf /backup/mysql-backup.tar.gz -C /
```

Part 2 — Docker Networking

Why Containers Need Networking

```
Without networking:
[Web App Container]    [Database Container]
        ↑                       ↑
Cannot reach each other — isolated by default

With Docker network:
[Web App Container] ←——— mynetwork ———→ [Database Container]
        Can communicate using container names
```

Three Built-in Network Types

```bash
docker network ls    # list all networks
```

```
NETWORK ID     NAME      DRIVER    SCOPE
a1b2c3d4e5f6   bridge    bridge    local
b2c3d4e5f6g7   host      host      local
c3d4e5f6g7h8   none      null      local
```

Bridge Network — Default

When you run a container without specifying a network — it joins the default bridge network.

```bash
```

```bash
# Both on default bridge:
docker run -d --name app1 nginx
docker run -d --name app2 nginx

# app1 and app2 can communicate by IP address
# BUT NOT by container name on default bridge
docker inspect app1 | grep IPAddress   # find IP
```

Problem with default bridge: Containers cannot reach each other by name — only by IP. IP addresses change — not reliable.

Host Network — Use Host Networking Directly

Container shares the host machine's network stack completely. No port mapping needed — container uses host ports directly.

```bash
docker run -d --network host nginx
# nginx now listens on host port 80 directly
# curl http://localhost:80 works without -p flag
```

Use case	Notes
High performance networking	No NAT overhead
Linux only	Does not work on Mac or Windows Docker
Port conflicts possible	Container uses host ports directly

None Network — Complete Isolation

Container has no network access at all.

```bash
docker run -d --network none myapp
# Cannot reach internet, cannot be reached
# Use for: batch processing, security-sensitive workloads
```

Custom Bridge Network — The Right Way

Custom networks solve the container name resolution problem. Containers on the same custom network can find each other **by container name.**

```bash
```

```bash
# Create custom network:
docker network create myapp-network

# Run containers on that network:
docker run -d \
  --name webapp \
  --network myapp-network \
  nginx

docker run -d \
  --name database \
  --network myapp-network \
  mysql:8.0 -e MYSQL_ROOT_PASSWORD=pass

# webapp can now reach database using hostname "database":
docker exec webapp curl http://database:3306
docker exec webapp ping database    # resolves by name!
```

Network Commands

```bash
docker network create myapp-network          # create network
docker network create --driver bridge mynet  # explicit driver
docker network ls                            # list all networks
docker network inspect myapp-network         # detailed info
docker network rm myapp-network              # remove network
docker network prune                         # remove unused networks

# Connect/disconnect running container:
docker network connect myapp-network mycontainer
docker network disconnect myapp-network mycontainer
```

Inspecting Network

```bash
docker network inspect myapp-network
```

Shows:

- All containers connected to this network
- IP addresses assigned to each
- Gateway and subnet information
- Driver configuration

Putting It Together — Real Multi-Container Application

Scenario — Web App + Database + Nginx Proxy

```
Internet → [Nginx Proxy :80] → [Flask App :5000] → [MySQL :3306]
                                        ↓
                                  [Volume: app-data]
```

Let's build this step by step:

Step 1 — Create Network

```bash
docker network create webapp-network
```

Step 2 — Start Database with Volume

```bash
docker run -d \
  --name mysql-db \
  --network webapp-network \
  -e MYSQL_ROOT_PASSWORD=secretpass \
  -e MYSQL_DATABASE=myapp \
  -e MYSQL_USER=appuser \
  -e MYSQL_PASSWORD=apppass \
  -v mysql-data:/var/lib/mysql \
  mysql:8.0
```

Step 3 — Start Application

```bash
docker run -d \
  --name flask-app \
  --network webapp-network \
  -e DB_HOST=mysql-db \
  -e DB_USER=appuser \
  -e DB_PASSWORD=apppass \
  -e DB_NAME=myapp \
```

```
  -v app-uploads:/app/uploads \
  myflaskapp:1.0
```

Notice `DB_HOST=mysql-db` — the app reaches the database using the container name `mysql-db` as the hostname. This works because both are on `webapp-network`.

Step 4 — Start Nginx as Reverse Proxy

```bash
docker run -d \
  --name nginx-proxy \
  --network webapp-network \
  -p 80:80 \
  -v $(pwd)/nginx.conf:/etc/nginx/conf.d/default.conf:ro \
  nginx:alpine
```

`nginx.conf` on host:

```nginx
server {
    listen 80;
    location / {
        proxy_pass http://flask-app:5000;
    }
}
```

Nginx forwards all traffic to `flask-app:5000` — again using container name as hostname.

Step 5 — Verify Everything

```bash
docker ps                             # all 3 containers running?
docker network inspect webapp-network # all 3 on same network?
curl http://localhost                  # does the app respond?
docker logs flask-app                 # any errors?
```

Cleanup

```bash
docker stop nginx-proxy flask-app mysql-db
docker rm nginx-proxy flask-app mysql-db
docker network rm webapp-network
```

```bash
# Volumes kept — data preserved
docker volume ls                        # mysql-data and app-uploads still exist
```

Port Mapping Deep Dive

How Port Mapping Works

```bash
docker run -p 8080:80 nginx
#              |    |
#              |    └── Container port (nginx listens here)
#              └──────── Host port (you access this from outside)
```

```
Your browser → localhost:8080 → Docker → Container:80 → nginx
```

Multiple Port Mappings

```bash
docker run -d \
  -p 8080:80 \     # HTTP
  -p 8443:443 \    # HTTPS
  --name multi-port-app \
  nginx
```

Bind to Specific Interface

```bash
docker run -p 127.0.0.1:8080:80 nginx   # only localhost can access
docker run -p 0.0.0.0:8080:80 nginx     # anyone can access (default)
```

In production — bind sensitive services to `127.0.0.1` only. Expose only what needs to be public.

Viewing Port Mappings

```bash
docker port my-nginx             # show port mappings for container
```

```
docker ps                          # PORTS column shows mappings
```

Full Summary — Docker Day 3

Volume Commands

Command	What it does
`docker volume create name`	Create named volume
`docker volume ls`	List all volumes
`docker volume inspect name`	Detailed volume info
`docker volume rm name`	Delete volume and data
`docker volume prune`	Delete all unused volumes
`-v name:/container/path`	Attach named volume
`-v /host/path:/container/path`	Bind mount
`-v /host/path:/container/path:ro`	Read-only bind mount

Network Commands

Command	What it does
`docker network create name`	Create custom network
`docker network ls`	List all networks
`docker network inspect name`	Detailed network info
`docker network rm name`	Remove network
`docker network prune`	Remove unused networks
`docker network connect net container`	Connect container to network
`--network name`	Run container on specific network

Interview Questions — Docker Day 3

Q1. Why do containers lose data when stopped? Answer: Containers have a writable layer on top of the read-only image layers. When a container is removed this writable layer is

deleted. Data written inside the container lives in this layer — no container means no data. Volumes solve this by storing data outside the container lifecycle.

Q2. What is the difference between a named volume and a bind mount? Answer: Named volumes are managed by Docker — stored in /var/lib/docker/volumes/ — best for production data persistence. Bind mounts map a specific host directory into the container — best for development where you want live code changes without rebuilding. Named volumes are more portable and work consistently across OSes.

Q3. What is the difference between the default bridge network and a custom bridge network? Answer: On the default bridge network containers can only communicate by IP address — not by name. On a custom bridge network Docker provides automatic DNS resolution — containers find each other using container names as hostnames. Always create custom networks for multi-container applications.

Q4. How do containers on the same network communicate? Answer: Containers on the same custom Docker network communicate using container names as hostnames. Docker's internal DNS resolves container names to their IP addresses automatically. Example: a web app container can reach a database container at hostname "mysql-db" if both are on the same network.

Q5. What is a reverse proxy in Docker context? Answer: A container running nginx or similar that receives all incoming traffic and forwards it to application containers on the internal network. Only the proxy is exposed on host ports — application containers are not directly reachable from outside. Provides a single entry point, SSL termination and load balancing.

Q6. What happens to volume data when you remove a container? Answer: Volume data is NOT deleted when a container is removed — it persists independently. Only the container itself is deleted. The data remains in the volume and can be attached to a new container. You must explicitly run docker volume rm to delete volume data.

Q7. What does `-p 127.0.0.1:8080:80` do differently from `-p 8080:80`? Answer: `-p 8080:80` binds to all interfaces (0.0.0.0) — anyone on the network can access it. `-p 127.0.0.1:8080:80` binds only to localhost — only connections from the same machine are accepted. Use localhost binding for sensitive internal services that should not be publicly accessible.

Homework — Before Docker Day 4

```bash
mkdir ~/docker-multicontainer
cd ~/docker-multicontainer
```

1. Create a named volume: `docker volume create mydata`
2. Run a container, write data to the volume, stop and remove it
3. Run a new container with the same volume — verify data is there

4. Create a custom network: `docker network create mynet`
5. Run two containers on `mynet` and verify they can ping each other by name
6. Run nginx with a bind mount connecting `$(pwd)/html` to `/usr/share/nginx/html`
7. Edit `html/index.html` while nginx runs — verify changes appear at `localhost:8080`

Your Progress

```
Linux  ━━━━━━━━━━━━━━━━━━━━━━━━  ✅ COMPLETE
Git    ━━━━━━━━━━━━━━━━━━━━━━━  ✅ COMPLETE
Docker ━━━━━━━━━━━━━━━━━━━━━━━  Day 3 of 8
AWS    ━━━━━━━━━━━━━━━━━━━━━━━  (parallel chat)
```

Docker Day 4 we learn **Docker Compose** — defining and running multi-container applications with a single file and a single command. This is what replaces all those long `docker run` commands you just typed. 💪

Say **"Docker Day 4"** whenever you are ready!

Docker Notes So Far — Days 1, 2 & 3

Day 1 — Introduction & Core Concepts

What is Docker?

Docker is a platform that packages applications with everything they need — code, runtime, libraries, config — into a **container.** Runs identically on any machine. Solves "works on my machine" problem permanently.

Containers vs Virtual Machines

Feature	Virtual Machine	Container
Size	Several GB each	Tens of MB
Startup time	Minutes	Seconds
OS	Full OS per VM	Shares host OS kernel
Performance	More overhead	Near native

Three Key Concepts

Concept	What it is	Analogy
Image	Read-only template	Recipe / class definition
Container	Running instance of image	Dish made from recipe
Registry	Storage for images (Docker Hub)	App store for images

Core Commands — Day 1

```bash
docker run hello-world                  # run first container
docker run -d -p 8080:80 --name web nginx  # run nginx detached
docker ps                               # list running containers
docker ps -a                            # list ALL containers
docker stop container_name             # stop gracefully
docker rm container_name               # remove container
docker logs name                       # view logs
docker logs -f name                    # follow logs live
docker exec -it name bash              # shell inside container
docker images                          # list local images
docker pull nginx:1.24                 # download specific version
docker rmi image                       # remove image
docker inspect name                    # detailed JSON info
docker stats                           # live resource usage
docker system prune                    # clean up unused resources
```

Important Flags

Flag	Meaning
`-d`	Detached — run in background
`-p 8080:80`	Map host port 8080 to container port 80
`-it`	Interactive terminal
`--name`	Give container a custom name
`--rm`	Auto-delete container when stopped
`-e KEY=value`	Set environment variable

Day 2 — Dockerfile & Building Custom Images

Every Dockerfile Instruction

```dockerfile
FROM ubuntu:22.04           # base image — every Dockerfile starts here
LABEL maintainer="varun"    # metadata
ENV APP_PORT=8080           # environment variable — build + runtime
ARG VERSION=1.0             # build-time variable only
WORKDIR /app                # set working directory — creates if missing
COPY src dst                # copy files into image
ADD src.tar.gz /app/        # copy + auto-extract archives
RUN apt-get install -y nginx # execute command during build
EXPOSE 80                   # document which port app uses
VOLUME /app/data            # declare persistent mount point
USER appuser                # switch to non-root user
HEALTHCHECK --interval=30s CMD curl -f http://localhost/ || exit 1
CMD ["nginx", "-g", "daemon off;"]      # default command — overridable
ENTRYPOINT ["python3"]      # fixed command — args appended to this
```

CMD vs ENTRYPOINT

	CMD	ENTRYPOINT
Overridable	Yes	No — args appended
Use for	Default arguments	Fixed executable
Best practice	Combine both	Combine both

```dockerfile
ENTRYPOINT ["python3"]    # always runs python3
CMD ["app.py"]            # default — overridable
# docker run img           → python3 app.py
# docker run img test.py   → python3 test.py
```

Layer Caching — Golden Rule

Put things that change LEAST at the top. Things that change MOST at the bottom.

```dockerfile
# WRONG — npm install runs every time code changes:
COPY . /app/
RUN npm install

# CORRECT — npm install cached unless package.json changes:
COPY package*.json ./
RUN npm install
COPY . .
```

Multi-Stage Builds

```dockerfile
# Stage 1 — Build (large, has all tools):
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2 — Production (small, only what runs):
FROM node:18-alpine AS production
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/package*.json ./
RUN npm install --production
USER node
EXPOSE 3000
CMD ["node", "dist/app.js"]
```

Result: 150MB instead of 900MB.

.dockerignore

```
node_modules/     # don't send to Docker daemon
.git/             # not needed in image
.env              # NEVER put secrets in image
*.log
dist/
.DS_Store
```

Build Commands

```bash
docker build -t myapp:1.0 .             # build image
docker build --no-cache -t myapp .      # force full rebuild
docker build -f Dockerfile.prod .       # specific Dockerfile
docker history myapp                    # see all layers
```

Common Mistakes

Mistake	Fix
Running as root	Add `USER appuser` before CMD

Mistake	Fix
Secrets in ENV	Pass with `-e` at runtime
No .dockerignore	Always create one
Using `latest` tag	Pin specific version `node:18.17.0`
Wrong layer order	Dependencies before source code

Day 3 — Volumes & Networking

Three Storage Types

Type	What it is	When to use
Named Volume	Docker-managed on host	Production data persistence
Bind Mount	Map specific host folder	Development — live reload
tmpfs	Memory only	Sensitive temp data

Volume Commands

```bash
docker volume create mydata            # create named volume
docker volume ls                       # list all volumes
docker volume inspect mydata           # see where data lives on host
docker volume rm mydata                # delete volume AND data
docker volume prune                    # remove all unused volumes

# Attach in docker run:
-v mydata:/app/data                    # named volume
-v $(pwd)/html:/usr/share/nginx/html   # bind mount
-v /host/path:/container/path:ro       # read-only bind mount
```

Named Volume vs Bind Mount

Named Volume	Bind Mount
Docker manages location	You specify exact path
Best for production	Best for development
Works on any OS	Path must exist on host
Hard to edit from host	Easy to edit from host

Three Network Types

Network	What it is	When to use
bridge (default)	Default — containers communicate by IP only	Single container
host	Container uses host network directly	High performance
none	No network access	Complete isolation
Custom bridge	Containers communicate by name	Multi-container apps

Network Commands

```bash
docker network create myapp-net        # create custom network
docker network ls                      # list all networks
docker network inspect myapp-net       # detailed info
docker network rm myapp-net            # remove network
docker network prune                   # remove unused networks
docker network connect net container   # connect running container

# Attach in docker run:
--network myapp-net                    # run on specific network
```

Why Custom Networks Matter

```bash
# Default bridge — containers can NOT reach each other by name
docker run --name app1 nginx           # joins default bridge
docker run --name app2 nginx           # cannot ping app1 by name

# Custom network — automatic DNS by container name
docker network create mynet
docker run --name app1 --network mynet nginx
docker run --name app2 --network mynet nginx
docker exec app2 ping app1             # WORKS — resolves by name
```

Real Multi-Container Setup

```bash
docker network create webapp-network

# Database with persistent volume:
docker run -d \
  --name mysql-db \
```

```bash
  --network webapp-network \
  -e MYSQL_ROOT_PASSWORD=secretpass \
  -e MYSQL_DATABASE=myapp \
  -v mysql-data:/var/lib/mysql \
  mysql:8.0

# App connects to DB using container name as hostname:
docker run -d \
  --name flask-app \
  --network webapp-network \
  -e DB_HOST=mysql-db \
  myflaskapp:1.0

# Nginx proxy — only container exposed to outside:
docker run -d \
  --name nginx-proxy \
  --network webapp-network \
  -p 80:80 \
  nginx:alpine
```

Port Mapping

```bash
-p 8080:80                # host:container — accessible from anywhere
-p 127.0.0.1:8080:80      # localhost only — not publicly accessible
docker port container_name # view current port mappings
```

Complete Docker Command Reference — Days 1-3

```bash
# CONTAINERS
docker run image                        # create and start
docker run -d image                     # background
docker run -it image bash               # interactive shell
docker run -p 8080:80 image             # map ports
docker run --name name image            # custom name
docker run --rm image                   # auto-delete on stop
docker run --network net image          # custom network
docker run -v vol:/path image           # named volume
docker run -v $(pwd):/path image        # bind mount
docker run -e KEY=val image             # environment variable
docker ps                               # running containers
docker ps -a                            # all containers
docker stop name                        # stop gracefully
docker start name                       # start stopped
docker restart name                     # restart
```

```
docker rm name                          # remove stopped
docker rm -f name                       # force remove running
docker logs name                        # view logs
docker logs -f name                     # follow live
docker exec -it name bash               # shell inside
docker inspect name                     # detailed info
docker stats                            # live resource usage
docker top name                         # processes inside

# IMAGES
docker images                           # list local images
docker pull image:tag                   # download image
docker build -t name:tag .              # build from Dockerfile
docker build --no-cache -t name .       # force rebuild
docker rmi image                        # remove image
docker history image                    # see layers
docker image prune                      # remove unused

# VOLUMES
docker volume create name               # create volume
docker volume ls                        # list volumes
docker volume inspect name              # volume details
docker volume rm name                   # delete volume
docker volume prune                     # remove unused

# NETWORKS
docker network create name              # create network
docker network ls                       # list networks
docker network inspect name             # network details
docker network rm name                  # remove network
docker network connect net container    # connect container
docker network prune                    # remove unused

# SYSTEM
docker system df                        # disk usage
docker system prune                     # clean everything unused
docker system prune -a                  # clean all not running
```

That is everything from Docker Days 1, 2 and 3. Now let's go straight into Day 4! 💪

Docker Day 4 — Docker Compose

Why Docker Compose Exists

Look at what you typed in Day 3 to run a 3-container application:

```bash
docker network create webapp-network
docker run -d --name mysql-db --network webapp-network \
  -e MYSQL_ROOT_PASSWORD=secretpass \
  -e MYSQL_DATABASE=myapp \
  -v mysql-data:/var/lib/mysql \
  mysql:8.0
docker run -d --name flask-app --network webapp-network \
  -e DB_HOST=mysql-db \
  -v app-uploads:/app/uploads \
  myflaskapp:1.0
docker run -d --name nginx-proxy --network webapp-network \
  -p 80:80 \
  -v $(pwd)/nginx.conf:/etc/nginx/conf.d/default.conf:ro \
  nginx:alpine
```

That is 4 commands, easy to get wrong, impossible to share with teammates reliably, and you must remember the exact order.

Docker Compose replaces all of this with one file and one command.

```bash
docker compose up -d    # starts everything
docker compose down     # stops everything
```

What You Will Learn Today

- What Docker Compose is
- The `docker-compose.yml` file structure
- Every key in docker-compose explained
- Building and running with Compose
- Environment variables in Compose
- Depends on and health checks
- Multiple environments — dev vs production
- Real full-stack application with Compose

What is Docker Compose?

Docker Compose is a tool for **defining and running multi-container applications** using a YAML file. Everything that was separate `docker run` commands becomes a single `docker-compose.yml` file that can be version-controlled, shared and reproduced exactly.

```
docker-compose.yml
        ↓
docker compose up
        ↓
All containers start together
All networks created automatically
All volumes created automatically
All in correct order
```

Installing Docker Compose

```bash
# Modern Docker installations include Compose as a plugin:
docker compose version
# Docker Compose version v2.20.0

# Older standalone installation:
docker-compose --version    # note the hyphen
```

Modern Docker uses `docker compose` (space). Older versions used `docker-compose` (hyphen). Both work — modern is preferred.

The docker-compose.yml File

Basic Structure

```yaml
version: "3.9"          # Compose file format version

services:               # containers to run
  web:                  # service name — becomes container name
    image: nginx        # image to use
    ports:
      - "8080:80"

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: secretpass

volumes:                # named volumes to create
  mysql-data:
```

```yaml
networks:                # custom networks to create
  myapp-network:
```

Run it:

```bash
docker compose up -d
docker compose down
```

Complete Service Configuration

```yaml
services:
  myapp:
    # ── Image ──────────────────────────────────────
    image: nginx:alpine                # use existing image
    build: .                           # build from Dockerfile in current dir
    build:                             # build with options
      context: ./app
      dockerfile: Dockerfile.prod
      args:
        VERSION: "1.0.0"

    # ── Container name ─────────────────────────────
    container_name: my-nginx           # custom name (optional)

    # ── Ports ──────────────────────────────────────
    ports:
      - "8080:80"                      # host:container
      - "127.0.0.1:443:443"           # localhost only

    # ── Environment variables ──────────────────────
    environment:
      - DB_HOST=database
      - DB_PORT=5432
    env_file:
      - .env                           # load from .env file

    # ── Volumes ────────────────────────────────────
    volumes:
      - mydata:/app/data               # named volume
      - ./config:/app/config:ro        # bind mount read-only
      - ./logs:/app/logs               # bind mount read-write

    # ── Networks ───────────────────────────────────
    networks:
```

```yaml
    - myapp-network

  # ── Dependencies ──────────────────────────────
  depends_on:
    database:
      condition: service_healthy    # wait until healthy

  # ── Restart policy ────────────────────────────
  restart: always                   # always restart if crashes
  # restart: unless-stopped         # restart unless manually stopped
  # restart: on-failure             # only restart on failure
  # restart: "no"                   # never restart (default)

  # ── Resource limits ───────────────────────────
  deploy:
    resources:
      limits:
        cpus: "0.5"                 # max 50% of one CPU
        memory: 512M                # max 512MB RAM

  # ── Health check ──────────────────────────────
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost/"]
    interval: 30s
    timeout: 5s
    retries: 3
    start_period: 10s

  # ── Command override ──────────────────────────
  command: ["nginx", "-g", "daemon off;"]

  # ── Working directory ─────────────────────────
  working_dir: /app

  # ── User ──────────────────────────────────────
  user: "1000:1000"
```

Core Docker Compose Commands

Starting and Stopping

```bash
docker compose up                   # start all services (foreground)
docker compose up -d                # start all detached (background)
docker compose up --build           # build images then start
```

```bash
docker compose up --build -d          # build and start in background
docker compose up web                 # start only web service

docker compose down                   # stop and remove containers + network
docker compose down -v                # also remove volumes — DELETES DATA
docker compose down --rmi all         # also remove images

docker compose start                  # start stopped services
docker compose stop                   # stop without removing
docker compose restart                # restart all services
docker compose restart web            # restart specific service
```

Viewing Status and Logs

```bash
docker compose ps                     # list services and status
docker compose logs                   # logs from all services
docker compose logs -f                # follow all logs live
docker compose logs web               # logs from specific service
docker compose logs -f web            # follow specific service
docker compose logs --tail 50 web     # last 50 lines
```

Running Commands in Services

```bash
docker compose exec web bash          # shell inside running service
docker compose exec web ls /app       # run command in running service
docker compose run web bash           # start NEW container and run command
```

Difference:

- `exec` — runs in **existing running** container
- `run` — starts a **new** container just for that command

Scaling Services

```bash
docker compose up -d --scale web=3   # run 3 instances of web service
```

Building

```bash
docker compose build                  # build all services
docker compose build web              # build specific service
docker compose build --no-cache       # build without cache
```

Real Application — Full Stack with Compose

Project Structure

```
myapp/
├── docker-compose.yml
├── docker-compose.override.yml    # dev overrides
├── docker-compose.prod.yml        # production config
├── .env                           # secrets — not in git
├── app/
|   ├── Dockerfile
|   └── app.py
└── nginx/
    └── nginx.conf
```

Main `docker-compose.yml`

```yaml
version: "3.9"

services:

  # ── Database ──────────────────────────────────
  database:
    image: mysql:8.0
    container_name: myapp-db
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    volumes:
      - mysql-data:/var/lib/mysql
      - ./mysql/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    networks:
      - backend
```

```yaml
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s

  # ── Application ──────────────────────────────────
  app:
    build:
      context: ./app
      dockerfile: Dockerfile
    container_name: myapp-flask
    restart: unless-stopped
    environment:
      - DB_HOST=database
      - DB_PORT=3306
      - DB_NAME=${DB_NAME}
      - DB_USER=${DB_USER}
      - DB_PASSWORD=${DB_PASSWORD}
      - FLASK_ENV=${FLASK_ENV:-production}
    volumes:
      - app-uploads:/app/uploads
    networks:
      - backend
      - frontend
    depends_on:
      database:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 5s
      retries: 3

  # ── Nginx Proxy ──────────────────────────────────
  nginx:
    image: nginx:alpine
    container_name: myapp-nginx
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - ./nginx/certs:/etc/nginx/certs:ro
    networks:
      - frontend
    depends_on:
```

```yaml
      app:
        condition: service_healthy

# ── Volumes ──────────────────────────────────────
volumes:
  mysql-data:
    driver: local
  app-uploads:
    driver: local

# ── Networks ─────────────────────────────────────
networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
```

The `.env` File

```bash
# .env — never commit to Git
DB_ROOT_PASSWORD=supersecretroot
DB_NAME=myapp
DB_USER=appuser
DB_PASSWORD=appuserpass
FLASK_ENV=production
```

Compose automatically loads `.env` from the same directory. Variables are referenced in `docker-compose.yml` with `${VARIABLE_NAME}`.

Development Override — `docker-compose.override.yml`

Compose automatically merges `docker-compose.override.yml` on top of `docker-compose.yml` when you run `docker compose up`. Used for development-specific settings.

```yaml
# docker-compose.override.yml
version: "3.9"

services:
  app:
    build:
      target: development           # use dev stage of multi-stage build
    volumes:
      - ./app:/app                  # bind mount — live code reload
```

```yaml
    environment:
      - FLASK_ENV=development
      - FLASK_DEBUG=1
    ports:
      - "5000:5000"                 # expose app port directly in dev

  database:
    ports:
      - "3306:3306"                 # expose DB in dev for local tools
```

```bash
# Development — uses both files automatically:
docker compose up -d

# Production — only main file:
docker compose -f docker-compose.yml up -d
```

Production Config — `docker-compose.prod.yml`

```yaml
# docker-compose.prod.yml
version: "3.9"

services:
  app:
    image: myapp:${APP_VERSION}     # use pre-built image not build
    restart: always
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M

  database:
    restart: always
    deploy:
      resources:
        limits:
          memory: 1G
```

```bash
# Run with production config:
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

`depends_on` and Health Checks — Critical Detail

```yaml
# WRONG — depends_on by default only waits for container to START
# Not for it to be READY:
depends_on:
  - database     # database container started but MySQL not ready yet
                 # app crashes trying to connect

# CORRECT — wait for healthy status:
depends_on:
  database:
    condition: service_healthy

# Combined with healthcheck on database service:
database:
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Without `service_healthy` condition — your app often crashes on startup because the database container started but MySQL inside it is not ready yet. This is one of the most common Docker Compose mistakes.

Common Compose Patterns

Pattern 1 — Build Custom Image

```yaml
services:
  app:
    build: .                          # Dockerfile in current dir
    image: myapp:latest               # also tag the built image
```

Pattern 2 — Share Environment Variables Between Services

```yaml
# Use YAML anchors to avoid repetition:
x-common-env: &common-env
```

```yaml
  DB_HOST: database
  DB_PORT: 5432
  REDIS_HOST: redis

services:
  app:
    environment:
      <<: *common-env
      FLASK_ENV: production

  worker:
    environment:
      <<: *common-env
      WORKER_CONCURRENCY: 4
```

Pattern 3 — One-Off Commands

```bash
# Run database migrations without starting full app:
docker compose run --rm app python manage.py migrate

# Run tests:
docker compose run --rm app pytest

# Open database shell:
docker compose exec database mysql -u root -p
```

Pattern 4 — Watch Logs During Deployment

```bash
docker compose up -d --build          # rebuild and restart
docker compose logs -f                # watch all logs
# Ctrl+C to stop following — containers keep running
```

Full Summary — Docker Day 4

Docker Compose Commands

Command	What it does
`docker compose up -d`	Start all services detached

Command	What it does
`docker compose up --build -d`	Build then start
`docker compose down`	Stop and remove containers
`docker compose down -v`	Also remove volumes — DELETES DATA
`docker compose ps`	List service status
`docker compose logs -f`	Follow all logs
`docker compose logs -f service`	Follow specific service
`docker compose exec service bash`	Shell in running service
`docker compose run --rm service cmd`	One-off command
`docker compose build`	Build all images
`docker compose build --no-cache`	Force full rebuild
`docker compose restart service`	Restart specific service

Key `docker-compose.yml` Options

Option	What it does
`image:`	Use existing image
`build:`	Build from Dockerfile
`ports:`	Map host to container ports
`environment:`	Set env variables
`env_file:`	Load from .env file
`volumes:`	Mount volumes
`networks:`	Connect to networks
`depends_on:`	Start order and health wait
`restart:`	Restart policy
`healthcheck:`	Define health check
`deploy:`	Resource limits

Interview Questions — Docker Day 4

Q1. What is Docker Compose and why is it used? Answer: Docker Compose is a tool for defining and running multi-container applications using a YAML file. Instead of multiple long docker run commands it describes all services, networks and volumes in one file. Run with docker compose up — everything starts together in correct order. Makes complex applications reproducible and shareable.

Q2. What is the difference between docker compose up and docker compose run? Answer: docker compose up starts all services defined in the file and keeps them running. docker compose run starts a one-off container for a specific service to run a single command — like running tests or database migrations — then exits. Use run --rm to automatically remove the container after the command finishes.

Q3. What does `depends_on` do and what is its limitation? Answer: depends_on controls startup order — a service waits for listed services to start before itself starts. The limitation is that by default it only waits for the container to START — not for the application inside to be READY. Use `condition: service_healthy` combined with healthcheck to wait until the service is actually accepting connections.

Q4. How do you use environment variables in Docker Compose? Answer: Three ways — inline in environment key, from an env_file, or from shell environment. Compose automatically loads a .env file in the same directory. Reference them in docker-compose.yml with ${VARIABLE_NAME}. Never hardcode secrets directly in the YAML file — use .env which is in .gitignore.

Q5. What is the difference between docker compose down and docker compose stop? Answer: stop stops the containers but leaves them and their networks intact. down stops AND removes containers and networks. down -v also removes volumes — this deletes all persistent data. Use stop when you want to start again quickly. Use down for a clean reset.

Q6. What is docker-compose.override.yml used for? Answer: Compose automatically merges override.yml on top of docker-compose.yml when running docker compose up. Used for development-specific settings — bind mounts for live code reload, extra port exposure, debug settings — without polluting the main file. Production uses only the main file explicitly.

Q7. How do you rebuild images when code changes? Answer: docker compose up --build -d — the --build flag forces Compose to rebuild images before starting. Without it Compose uses cached images even if your Dockerfile or code changed. In production pipelines images are pre-built and pushed to a registry — docker compose pull then docker compose up.

Homework — Before Docker Day 5

```bash
mkdir ~/docker-compose-practice
cd ~/docker-compose-practice
```

1. Create a `docker-compose.yml` with nginx and a simple html page served via bind mount
2. Add a second service — any database you choose
3. Create a `.env` file for database credentials
4. Use `depends_on` with `service_healthy` condition
5. Run `docker compose up -d` and verify both services work
6. Run `docker compose logs -f` and watch the output
7. Run `docker compose down -v` and verify volumes are gone
8. Create `docker-compose.override.yml` with development settings

Your Progress

```
Linux   ▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰  ✅ COMPLETE
Git     ▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰  ✅ COMPLETE
Docker  ▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱▱▱▱▱   Day 4 of 8
AWS     ▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱  (parallel chat)
```

Docker Day 5 we learn **Docker Registry, Image Tagging and Pushing to Docker Hub and AWS ECR** — how to store and distribute your images professionally so servers can pull and run them. 💪

Say **"Docker Day 5"** whenever you are ready!

Where You Are Right Now

```
Linux   ▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰  ✅ COMPLETE — 16 days
Git     ▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰  ✅ COMPLETE — 4 days
Docker  ▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱▱▱▱▱   Day 4 of 8 done — 50%
AWS     ▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱  (parallel chat)
CI/CD   ▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱  After Docker
K8s     ▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱  After CI/CD
```

Docker Remaining Days

Day	Topic	Status
Day 1	Introduction, containers, core commands	✅
Day 2	Dockerfile, building images, multi-stage	✅
Day 3	Volumes and Networking	✅

Day	Topic	Status
Day 4	Docker Compose	✅
Day 5	Registry, tagging, Docker Hub, ECR	← Now
Day 6	Docker security best practices	Pending
Day 7	Real DevOps project — full pipeline	Pending
Day 8	Docker interview mega revision	Pending

After Docker Completes

Topic	Days Needed
CI/CD — Jenkins + GitHub Actions	5 days
Kubernetes basics	6 days
Interview mega prep (all topics)	2 days

Total Remaining

```
Docker remaining  4 days
CI/CD             5 days
Kubernetes        6 days
Final prep        2 days
──────────────────────────
Total remaining   17 days
```

At 1-2 hours/day you are roughly 3 weeks from being fully interview-ready. You are right on track for your 3-month goal. 💪

Now let's get into Day 5!

Docker Day 5 — Registry, Image Tagging & Pushing

Why This is Critical for DevOps

Right now your Docker images exist only on your laptop. In real DevOps:

- Your CI/CD pipeline builds an image on a build server
- That image needs to be deployed to 10 production servers

- Each server needs to pull the exact same image
- You need to keep track of versions — v1.0, v1.1, v2.0

A registry solves all of this. It is the central storage for your Docker images. Build once — push to registry — pull from anywhere.

This is a core step in every real CI/CD pipeline:

```
Code pushed to Git
      ↓
CI builds Docker image
      ↓
CI pushes image to registry
      ↓
Production servers pull image from registry
      ↓
Containers updated
```

What You Will Learn Today

- What a Docker registry is
- Docker Hub — public registry
- Image naming and tagging properly
- Pushing images to Docker Hub
- Pulling images from Docker Hub
- Private registries
- AWS ECR — Elastic Container Registry
- Image tagging strategy for production
- Registry in CI/CD pipeline

What is a Docker Registry?

A registry is a **storage and distribution system for Docker images.** Think of it like GitHub — but for Docker images instead of code.

```
Developer laptop          Registry              Production Server
                       (Docker Hub / ECR)
docker build  ──push──>  stores image  ──pull──>  docker run
```

Registry	Type	Use case
Docker Hub	Public cloud	Open source, personal projects

Registry	Type	Use case
AWS ECR	Private cloud	AWS-based production workloads
GitHub Container Registry	Private cloud	GitHub-integrated workflows
GitLab Registry	Private cloud	GitLab CI/CD workflows
Self-hosted Registry	Private on-premise	Air-gapped environments

Image Naming — The Full Format

```
registry/username/image-name:tag

docker.io/varun/myapp:1.0.0
    |       |      |     |
    |       |      |     └── Tag (version)
    |       |      └──────── Image name
    |       └───────────────── Username / organisation
    └──────────────────────── Registry (docker.io = Docker Hub default)
```

When you run `docker pull nginx` Docker expands it to `docker.io/library/nginx:latest` automatically.

Docker Hub

Setting Up Docker Hub

1. Go to hub.docker.com
2. Create free account
3. Note your username — you need it for all image names

Logging In to Docker Hub

```bash
docker login
# Prompts for username and password
# Credentials saved in ~/.docker/config.json

docker login -u varun
# Prompts for password only
```

```bash
docker logout
# Removes stored credentials
```

Tagging Images Properly

Before pushing to Docker Hub your image must be named correctly:

```bash
# Format: username/imagename:tag
docker tag myapp varun/myapp:1.0.0
docker tag myapp varun/myapp:latest

# Tag existing image with full registry path:
docker tag myapp docker.io/varun/myapp:1.0.0

# View all tags on local images:
docker images | grep myapp
```

Pushing to Docker Hub

```bash
# Push specific tag:
docker push varun/myapp:1.0.0

# Push latest tag:
docker push varun/myapp:latest

# Push all tags for an image:
docker push varun/myapp --all-tags
```

Output:

```
The push refers to repository [docker.io/varun/myapp]
a1b2c3d4e5f6: Pushed
b2c3d4e5f6g7: Layer already exists
1.0.0: digest: sha256:abc123... size: 1234
```

"Layer already exists" means Docker is reusing cached layers — only new layers are uploaded. Efficient.

Pulling from Docker Hub

```bash
```

```bash
# Pull specific version:
docker pull varun/myapp:1.0.0

# Pull latest:
docker pull varun/myapp

# Pull and run in one step:
docker run varun/myapp:1.0.0
```

Full Docker Hub Workflow

```bash
# Step 1 — Build your image:
docker build -t myapp .

# Step 2 — Tag it with Docker Hub username:
docker tag myapp varun/myapp:1.0.0
docker tag myapp varun/myapp:latest

# Step 3 — Login:
docker login

# Step 4 — Push:
docker push varun/myapp:1.0.0
docker push varun/myapp:latest

# Step 5 — On any other machine pull and run:
docker pull varun/myapp:1.0.0
docker run -d -p 8080:80 varun/myapp:1.0.0
```

Image Tagging Strategy — Production Best Practice

The Problem with `latest`

```bash
docker pull varun/myapp:latest     # which version is this exactly?
```

`latest` is just a tag — it does not automatically update. It means "the last thing someone pushed with the latest tag." In production you never know exactly what version you have.

Real production systems never use `latest` in deployments.

Semantic Versioning Tags

```bash
# Tag with semantic version:
docker tag myapp varun/myapp:1.2.3

# Also tag with minor version (auto-gets latest patch):
docker tag myapp varun/myapp:1.2

# Also tag with major version (auto-gets latest minor):
docker tag myapp varun/myapp:1

# Also tag as latest:
docker tag myapp varun/myapp:latest

# Push all:
docker push varun/myapp:1.2.3
docker push varun/myapp:1.2
docker push varun/myapp:1
docker push varun/myapp:latest
```

Now:

- Production uses `varun/myapp:1.2.3` — exact, never changes
- Staging uses `varun/myapp:1.2` — gets patches automatically
- Development uses `varun/myapp:latest` — always newest

Git SHA Tagging — Most Reliable for CI/CD

```bash
# In CI/CD pipeline — tag with Git commit SHA:
GIT_SHA=$(git rev-parse --short HEAD)    # e.g. a1b2c3d

docker build -t myapp .
docker tag myapp varun/myapp:${GIT_SHA}
docker push varun/myapp:${GIT_SHA}

# Deploy to production with exact SHA:
docker run varun/myapp:a1b2c3d
```

Every image is traceable back to exact commit. If production breaks — you know exactly which code is running.

Combined Tagging Strategy — Used in Real Companies

```bash
GIT_SHA=$(git rev-parse --short HEAD)
VERSION="1.2.3"
BRANCH=$(git branch --show-current)

docker build -t myapp .

# Tag with multiple strategies:
docker tag myapp varun/myapp:${VERSION}      # semantic version
docker tag myapp varun/myapp:${GIT_SHA}      # git SHA — traceable
docker tag myapp varun/myapp:${BRANCH}       # branch name
docker tag myapp varun/myapp:latest          # latest

# Push all:
for tag in ${VERSION} ${GIT_SHA} ${BRANCH} latest; do
    docker push varun/myapp:${tag}
done
```

AWS ECR — Elastic Container Registry

What is ECR?

AWS ECR is Amazon's private Docker registry. If you deploy on AWS — ECR is the standard choice because:

- Integrated with AWS IAM — no separate credentials
- Integrated with ECS, EKS, CodePipeline
- High availability and low latency within AWS
- Automatic image scanning for vulnerabilities

ECR Repository Naming

```
aws_account_id.dkr.ecr.region.amazonaws.com/repository-name:tag

123456789012.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0.0
```

Pushing to ECR — Step by Step

```bash
```

```
# Step 1 — Create repository in ECR (AWS Console or CLI):
aws ecr create-repository \
    --repository-name myapp \
    --region ap-south-1

# Step 2 — Authenticate Docker to ECR:
aws ecr get-login-password --region ap-south-1 | \
    docker login \
    --username AWS \
    --password-stdin \
    123456789012.dkr.ecr.ap-south-1.amazonaws.com

# Step 3 — Tag image with ECR URI:
docker tag myapp:latest \
    123456789012.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0.0

# Step 4 — Push:
docker push \
    123456789012.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0.0

# Step 5 — Pull from ECR (on any AWS machine with correct IAM role):
docker pull \
    123456789012.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0.0
```

ECR Lifecycle Policy — Auto-Cleanup Old Images

Without cleanup ECR fills up and costs money. Set lifecycle policies:

```json
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Keep only last 10 images",
      "selection": {
        "tagStatus": "any",
        "countType": "imageCountMoreThan",
        "countNumber": 10
      },
      "action": {
        "type": "expire"
      }
    }
  ]
}
```

```bash
aws ecr put-lifecycle-policy \
    --repository-name myapp \
    --lifecycle-policy-text file://lifecycle.json
```

Running a Private Registry

Self-Hosted Registry

For air-gapped environments or when you don't want to use cloud:

```bash
# Run Docker's official registry image:
docker run -d \
    --name registry \
    -p 5000:5000 \
    -v registry-data:/var/lib/registry \
    registry:2

# Tag for local registry:
docker tag myapp localhost:5000/myapp:1.0.0

# Push to local registry:
docker push localhost:5000/myapp:1.0.0

# Pull from local registry:
docker pull localhost:5000/myapp:1.0.0
```

Registry in CI/CD Pipeline

How Images Flow in a Real Pipeline

```
Developer pushes code to GitHub
              ↓
GitHub Actions / Jenkins starts
              ↓
CI checks out code
              ↓
docker build -t myapp:${GIT_SHA} .
              ↓
docker push registry/myapp:${GIT_SHA}
              ↓
```

```
Deploy job pulls image on production server
              ↓
docker pull registry/myapp:${GIT_SHA}
docker run registry/myapp:${GIT_SHA}
              ↓
Old container replaced — zero downtime
```

GitHub Actions Example — Build and Push

```yaml
# .github/workflows/docker.yml
name: Build and Push Docker Image

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Login to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and push
        uses: docker/build-push-action@v4
        with:
          context: .
          push: true
          tags: |
            varun/myapp:latest
            varun/myapp:${{ github.sha }}
```

Every push to main automatically builds and pushes a new image. This is the CI part of CI/CD.

Jenkins Pipeline Example

```groovy
```

```
pipeline {
    agent any
    environment {
        REGISTRY = "varun"
        IMAGE = "myapp"
        TAG = "${GIT_COMMIT[0..6]}"
    }
    stages {
        stage('Build') {
            steps {
                sh "docker build -t ${REGISTRY}/${IMAGE}:${TAG} ."
            }
        }
        stage('Push') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub',
                    usernameVariable: 'USER',
                    passwordVariable: 'PASS'
                )]) {
                    sh "echo $PASS | docker login -u $USER --password-stdin"
                    sh "docker push ${REGISTRY}/${IMAGE}:${TAG}"
                }
            }
        }
        stage('Deploy') {
            steps {
                sh "docker pull ${REGISTRY}/${IMAGE}:${TAG}"
                sh "docker stop myapp || true"
                sh "docker rm myapp || true"
                sh "docker run -d --name myapp -p 80:80 ${REGISTRY}/${IMAGE}:${TAG}"
            }
        }
    }
}
```

Useful Registry Commands Summary

```bash
# Docker Hub:
docker login                          # login to Docker Hub
docker logout                         # logout
docker pull image:tag                 # pull image
docker push username/image:tag        # push image
docker search nginx                   # search Docker Hub
```

```
# ECR:
aws ecr create-repository --repository-name myapp --region ap-south-1
aws ecr get-login-password | docker login --username AWS --password-stdin URI
aws ecr describe-repositories        # list ECR repos
aws ecr list-images --repository-name myapp   # list images in repo
aws ecr batch-delete-image --repository-name myapp --image-ids imageTag=old

# Local registry:
docker run -d -p 5000:5000 --name registry registry:2
docker tag myapp localhost:5000/myapp:1.0.0
docker push localhost:5000/myapp:1.0.0
```

Full Summary — Docker Day 5

Concept	Key points
Registry	Central storage for Docker images
Docker Hub	Public registry — free for public images
ECR	AWS private registry — integrated with AWS services
Image naming	registry/username/name:tag
Tagging strategy	Use semantic versions + git SHA — never rely on latest alone
CI/CD flow	build → tag → push → pull → run

Interview Questions — Docker Day 5

Q1. What is a Docker registry? Answer: A registry is a storage and distribution system for Docker images — like GitHub but for images. Docker Hub is the default public registry. AWS ECR is a popular private registry for production workloads. Images are pushed to the registry from CI and pulled by production servers.

Q2. What is the problem with using the `latest` tag in production? Answer: latest is just a tag — it does not automatically update and does not tell you which version is actually running. Two servers pulling latest at different times may get different images. In production always use specific version tags like semantic versions or git commit SHAs so you know exactly what is deployed.

Q3. What is the difference between Docker Hub and AWS ECR? Answer: Docker Hub is a public cloud registry — images can be public or private, free tier has limits. AWS ECR is a private registry integrated with AWS — uses IAM for authentication, integrates natively with ECS and EKS, supports image vulnerability scanning. ECR is preferred for production workloads on AWS.

Q4. How does image tagging work in a CI/CD pipeline? Answer: CI builds the image then tags it with the git commit SHA for traceability plus a semantic version. Both tags are pushed to the registry. Deployment jobs pull the exact SHA tag ensuring they get the right version. Example: docker tag myapp registry/myapp:a1b2c3d then docker push registry/myapp:a1b2c3d.

Q5. How do you authenticate to AWS ECR? Answer: Using the AWS CLI to get a temporary token then pipe it to docker login. Command: `aws ecr get-login-password --region region | docker login --username AWS --password-stdin account.dkr.ecr.region.amazonaws.com`. The token is valid for 12 hours. CI/CD systems do this automatically using IAM roles.

Q6. What happens to layers when pushing an image? Answer: Docker only pushes layers that do not already exist in the registry. If a layer is already there it shows "Layer already exists" and skips it. This makes pushes very fast when only a small part of the image changed — only new layers are uploaded.

Homework — Before Docker Day 6

1. Create a Docker Hub account if you don't have one
2. Build any simple image from your Day 2 Dockerfile
3. Tag it with your Docker Hub username: `docker tag myapp yourusername/myapp:1.0.0`
4. Login and push: `docker login && docker push yourusername/myapp:1.0.0`
5. Go to hub.docker.com and verify your image is there
6. Pull it on the same machine after removing local copy:

```bash
docker rmi yourusername/myapp:1.0.0
docker pull yourusername/myapp:1.0.0
```

7. Write a shell script that builds, tags with git SHA and pushes automatically

---

Your Progress

```
Linux   ▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰ ✅ COMPLETE
Git     ▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰ ✅ COMPLETE
Docker  ▰▰▰▰▰▰▰▰▰▰▱▱▱▱▱▱▱▱▱▱  Day 5 of 8 — 62%
AWS     ▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱ (parallel chat)
CI/CD   ▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱ After Docker
K8s     ▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱▱ After CI/CD
```

Docker Day 6 we cover **Docker Security Best Practices** — the topic that separates junior from senior Docker knowledge and comes up heavily in 10-12 LPA interviews. 💪

Say **"Docker Day 6"** whenever you are ready!

Docker Day 6 — Security Best Practices

Why Security is Critical for DevOps

At 10-12 LPA level interviewers do not just ask "how do you run a container." They ask:

- "How do you make sure your containers are secure?"
- "What happens if a container is compromised?"
- "How do you prevent secrets from leaking?"
- "What is the principle of least privilege in Docker?"

Companies that deploy Docker in production have been breached because of misconfigured containers. A container running as root, exposed Docker socket, hardcoded secrets — these are real attack vectors that have caused real incidents.

Security is not optional in DevOps. It is part of every deployment.

What You Will Learn Today

- Why containers running as root is dangerous
- User namespaces and non-root users
- Read-only filesystems
- Secrets management — the right way
- Limiting container capabilities
- Resource limits — preventing DoS
- Image scanning for vulnerabilities
- Docker socket security
- Network security
- Security checklist for production

The Attacker's Goal

Before security — understand what an attacker wants:

```
Attack path without security:
```

```
1. Find vulnerability in your web app
2. Execute code inside container
3. Container runs as root
4. Break out of container to host
5. Root on host = full server compromise
6. Access all other containers
7. Access cloud credentials
8. Access your entire infrastructure
```

Every security practice today cuts off one of these steps.

Security Practice 1 — Never Run as Root

Why Root in Container is Dangerous

```bash
# Check who you are inside a default container:
docker run --rm ubuntu whoami
# root

# This root IS the same UID 0 as host root
# If container escapes — attacker has root on host
```

Most official Docker images run as root by default. This is wrong for production.

Creating Non-Root Users in Dockerfile

```dockerfile
FROM ubuntu:22.04

# Install dependencies as root first
RUN apt-get update && \
    apt-get install -y curl && \
    rm -rf /var/lib/apt/lists/*

# Create dedicated app user
RUN groupadd -r appgroup && \
    useradd -r -g appgroup -s /bin/false -d /app appuser

# Create app directory and set ownership
RUN mkdir -p /app && chown -R appuser:appgroup /app

WORKDIR /app
```

```dockerfile
# Copy files
COPY --chown=appuser:appgroup . .

# Switch to non-root user
USER appuser

# Now everything runs as appuser — not root
CMD ["./myapp"]
```

Verify Container is Not Running as Root

```bash
docker run --rm myapp whoami
# appuser   ← correct

docker run --rm myapp id
# uid=1001(appuser) gid=1001(appgroup) groups=1001(appgroup)
```

Force Non-Root at Runtime

```bash
# Run as specific user even if image uses root:
docker run --user 1001:1001 nginx

# Run as nobody:
docker run --user nobody nginx
```

⚠️ Common Problem — File Permission Errors

```dockerfile
# Wrong — app cannot write to /app/logs:
RUN mkdir /app/logs
USER appuser
# appuser does not own /app/logs — cannot write

# Correct — set ownership before switching:
RUN mkdir /app/logs && chown appuser:appgroup /app/logs
USER appuser
# Now appuser owns the directory
```

Security Practice 2 — Read-Only Filesystem

Why Read-Only Matters

If an attacker gets code execution in your container — they often need to write files. A read-only filesystem prevents them from:

- Installing tools (wget, curl, nc)
- Writing backdoors
- Modifying application files

```bash
# Run container with read-only root filesystem:
docker run --read-only nginx

# Error: nginx cannot write its temp files
# Fix — allow writes only to specific temp directories:
docker run \
  --read-only \
  --tmpfs /tmp \
  --tmpfs /var/cache/nginx \
  --tmpfs /var/run \
  nginx
```

Read-Only in Docker Compose

```yaml
services:
  app:
    image: myapp:1.0
    read_only: true
    tmpfs:
      - /tmp
      - /app/cache
    volumes:
      - app-logs:/app/logs       # only this directory writable
```

Security Practice 3 — Secrets Management

The Wrong Ways to Handle Secrets

```dockerfile
# WRONG 1 — hardcoded in Dockerfile:
ENV DB_PASSWORD=supersecret123

# WRONG 2 — passed in RUN command (visible in docker history):
RUN curl -u admin:supersecret https://api.example.com

# WRONG 3 — copied into image:
COPY .env /app/.env
```

All of these embed secrets into the image layers permanently. Anyone with access to the image can read them.

```bash
docker history myapp --no-trunc    # reveals secrets in RUN commands
docker inspect myapp               # reveals ENV variables
```

The Right Ways to Handle Secrets

Method 1 — Environment Variables at Runtime

```bash
# Pass at runtime — not baked into image:
docker run -e DB_PASSWORD=secretpass myapp

# From .env file — not committed to Git:
docker run --env-file .env myapp
```

Method 2 — Docker Secrets (Swarm)

```bash
# Create secret:
echo "supersecretpassword" | docker secret create db_password -

# Use in service:
docker service create \
  --name myapp \
  --secret db_password \
  myapp:1.0

# Inside container — secret available at:
cat /run/secrets/db_password
```

Method 3 — Mount Secrets as Files

```bash
# Create a secrets directory on host:
echo "supersecretpassword" > /run/secrets/db_password
chmod 600 /run/secrets/db_password

# Mount into container read-only:
docker run \
  -v /run/secrets:/run/secrets:ro \
  myapp

# Application reads from file:
DB_PASSWORD=$(cat /run/secrets/db_password)
```

Method 4 — AWS Secrets Manager (Production)

```python
# Application fetches secret at runtime from AWS:
import boto3

client = boto3.client('secretsmanager')
secret = client.get_secret_value(SecretId='prod/myapp/db')
db_password = secret['SecretString']
```

No secret ever in code, Dockerfile or environment variables. Fetched at runtime using IAM role permissions.

Multi-Stage Build — Remove Secrets After Build

```dockerfile
# Stage 1 — Build (needs secrets):
FROM node:18 AS builder
ARG NPM_TOKEN
RUN echo "//registry.npmjs.org/:_authToken=${NPM_TOKEN}" > .npmrc
RUN npm install
RUN rm -f .npmrc              # remove before next stage

# Stage 2 — Production (no secrets):
FROM node:18-alpine
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
```

```
# .npmrc never makes it here
CMD ["node", "dist/app.js"]
```

```bash
docker build \
  --build-arg NPM_TOKEN=mytoken \
  -t myapp .
# Token used during build, never stored in final image
```

Security Practice 4 — Linux Capabilities

What are Capabilities?

Linux root is not all-or-nothing. It is divided into ~40 capabilities:

Capability	What it allows
`CAP_NET_ADMIN`	Configure network interfaces
`CAP_SYS_ADMIN`	Mount filesystems, many system calls
`CAP_CHOWN`	Change file ownership
`CAP_NET_BIND_SERVICE`	Bind to ports below 1024
`CAP_KILL`	Send signals to any process

By default Docker gives containers a large set of capabilities — more than most apps need.

Drop All Capabilities — Add Only What You Need

```bash
# Drop ALL capabilities then add only what app needs:
docker run \
  --cap-drop ALL \
  --cap-add NET_BIND_SERVICE \
  nginx

# Now nginx can bind to port 80 but cannot do anything else privileged
```

In Docker Compose

```yaml
services:
  nginx:
    image: nginx:alpine
    cap_drop:
      - ALL
    cap_add:
      - NET_BIND_SERVICE
      - CHOWN
      - SETUID
      - SETGID
```

The `--privileged` Flag — Never in Production

```bash
docker run --privileged myapp    # gives ALL capabilities + host device access
```

`--privileged` essentially removes all container isolation. The container can mount host filesystems, load kernel modules and do almost anything. **Never use in production.** Only acceptable for very specific dev/testing scenarios.

Security Practice 5 — Resource Limits

Why Resource Limits Matter

Without limits a single container can consume all CPU and memory on a host — starving other containers and crashing the server. This is called a Denial of Service.

A compromised container running a crypto miner will consume 100% CPU without limits.

Memory Limits

```bash
# Limit to 512MB — container killed if exceeds:
docker run \
  --memory 512m \
  --memory-swap 512m \    # same as memory = no swap
  myapp

# Soft limit — warning only:
docker run --memory-reservation 256m myapp
```

CPU Limits

```bash
# Limit to 50% of one CPU core:
docker run --cpus="0.5" myapp

# Limit to 2 CPU cores:
docker run --cpus="2" myapp

# CPU shares — relative weight:
docker run --cpu-shares 512 myapp    # half of default 1024
```

In Docker Compose

```yaml
services:
  app:
    image: myapp:1.0
    deploy:
      resources:
        limits:
          cpus: "0.5"
          memory: 512M
        reservations:
          cpus: "0.25"
          memory: 256M
```

Checking Resource Usage

```bash
docker stats                          # live usage all containers
docker stats myapp                    # specific container
docker inspect myapp | grep -i memory  # configured limits
```

Security Practice 6 — Image Vulnerability Scanning

Why Scan Images?

Base images like `ubuntu:22.04` contain hundreds of packages. Many have known vulnerabilities — CVEs (Common Vulnerabilities and Exposures). If your image has a critical CVE and attackers know about it — they can exploit it.

Docker Scout — Built-in Scanning

```bash
# Scan image for vulnerabilities:
docker scout cves myapp:1.0

# Quick summary:
docker scout quickview myapp:1.0

# Compare with previous version:
docker scout compare myapp:1.0 myapp:0.9
```

Trivy — Popular Open Source Scanner

```bash
# Install trivy:
sudo apt install -y trivy

# Scan local image:
trivy image myapp:1.0

# Scan with severity filter:
trivy image --severity HIGH,CRITICAL myapp:1.0

# Scan and fail CI if critical found:
trivy image --exit-code 1 --severity CRITICAL myapp:1.0
```

Output shows:

```
myapp:1.0 (ubuntu 22.04)
==================================================
Total: 23 (HIGH: 5, CRITICAL: 2)

Package          Vulnerability     Severity     Fixed Version
openssl          CVE-2023-1234     CRITICAL     1.1.1t
libssl           CVE-2023-5678     HIGH         1.1.1s
```

Reduce Attack Surface — Use Minimal Base Images

```dockerfile
# Large base — many vulnerabilities:
FROM ubuntu:22.04            # 400+ packages, many CVEs

# Smaller — fewer vulnerabilities:
FROM ubuntu:22.04-minimal    # much smaller

# Minimal — very few packages:
FROM alpine:3.18             # ~5MB, very few CVEs

# Distroless — no shell, no package manager:
FROM gcr.io/distroless/python3    # only Python runtime
# Attacker cannot run shell commands even if they get in
```

Keep Images Updated

```bash
# Always build with latest base image:
docker build --pull -t myapp .     # --pull always fetches latest base

# In CI/CD — rebuild weekly even without code changes:
# Scheduled pipeline rebuilds image to get security patches
```

Security Practice 7 — Docker Socket Security

The Most Dangerous Misconfiguration

The Docker socket `/var/run/docker.sock` gives **full control of Docker** on the host. Mounting it into a container is essentially giving that container root on the host.

```bash
# EXTREMELY DANGEROUS — never do this in production:
docker run -v /var/run/docker.sock:/var/run/docker.sock myapp

# If container is compromised:
docker run -it \
  -v /var/run/docker.sock:/var/run/docker.sock \
  ubuntu bash
# Inside container:
docker run -it --privileged -v /:/host ubuntu bash
# Now you have full host root access
```

When is Docker Socket Needed?

Some legitimate tools need it — CI/CD agents, monitoring tools like Portainer. For these:

```bash
# Use Docker-in-Docker (dind) instead:
docker run \
  --privileged \
  docker:dind

# Or use rootless Docker
# Or use a Docker socket proxy that limits access
```

Security Practice 8 — Network Security

Isolate Services on Separate Networks

```yaml
services:
  nginx:
    networks:
      - frontend          # only on public-facing network

  app:
    networks:
      - frontend          # talks to nginx
      - backend           # talks to database

  database:
    networks:
      - backend           # only on internal network — not reachable from nginx dire

networks:
  frontend:
  backend:
    internal: true        # backend network has no internet access
```

`internal: true` means containers on that network cannot reach the internet. Database cannot make outbound connections — limits blast radius if compromised.

Disable Inter-Container Communication

```bash
# On default bridge network — prevent all container-to-container communication:
dockerd --icc=false

# In daemon.json:
{
    "icc": false
}
```

Now containers can only communicate through explicitly defined network links.

Security Checklist — Production Ready

The Complete Checklist

```bash
# 1. Non-root user in Dockerfile:
USER appuser                          ✅

# 2. Read-only filesystem:
--read-only                           ✅

# 3. No secrets in image:
docker history myimage | grep -i pass ✅  # should show nothing

# 4. Minimal base image:
FROM alpine OR distroless             ✅

# 5. Drop capabilities:
--cap-drop ALL --cap-add only-needed  ✅

# 6. Resource limits set:
--memory 512m --cpus 0.5              ✅

# 7. Image scanned for CVEs:
trivy image myapp --severity CRITICAL ✅

# 8. No Docker socket mounted:
grep -r docker.sock docker-compose.yml ✅  # should show nothing

# 9. Network isolation:
internal: true on backend networks    ✅
```

```
# 10. No --privileged flag:
grep privileged docker-compose.yml     ✅  # should show nothing
```

Secure Docker Compose Example

```yaml
version: "3.9"

services:
  app:
    image: myapp:1.2.3              # pinned version — not latest
    container_name: myapp
    user: "1001:1001"               # non-root user
    read_only: true                 # read-only filesystem
    tmpfs:
      - /tmp                        # allow writes only to /tmp
    cap_drop:
      - ALL                         # drop all capabilities
    cap_add:
      - NET_BIND_SERVICE            # add only what is needed
    security_opt:
      - no-new-privileges:true      # prevent privilege escalation
    environment:
      - DB_HOST=database
    env_file:
      - .env                        # secrets from env file not in YAML
    networks:
      - frontend
    deploy:
      resources:
        limits:
          cpus: "0.5"
          memory: 256M
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 5s
      retries: 3
    restart: unless-stopped

  database:
    image: mysql:8.0.33             # pinned version
    user: "999:999"                 # mysql user
    read_only: true
    tmpfs:
      - /tmp
      - /var/run/mysqld
```

```yaml
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
    environment:
      MYSQL_ROOT_PASSWORD_FILE: /run/secrets/db_root_pass
    volumes:
      - mysql-data:/var/lib/mysql   # data persists
      - /run/secrets:/run/secrets:ro  # secrets read-only
    networks:
      - backend                     # only on backend — not reachable from internet
    deploy:
      resources:
        limits:
          memory: 1G

volumes:
  mysql-data:

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
    internal: true                  # no internet access
```

`security_opt: no-new-privileges`

```yaml
security_opt:
  - no-new-privileges:true
```

Prevents any process inside the container from gaining new privileges through setuid binaries or sudo. Even if an attacker finds a setuid binary — they cannot escalate.

Full Summary — Docker Day 6

Practice	Command / Config	Why
Non-root user	`USER appuser` in Dockerfile	Limits damage if container compromised
Read-only filesystem	`--read-only`	Prevents writing backdoors

Practice	Command / Config	Why
No secrets in image	`-e` at runtime or `--env-file`	Secrets not baked into layers
Drop capabilities	`--cap-drop ALL`	Principle of least privilege
Resource limits	`--memory 512m --cpus 0.5`	Prevent DoS and crypto mining
Image scanning	`trivy image myapp`	Find known CVEs before deployment
No Docker socket	Never `-v /var/run/docker.sock`	Prevents full host compromise
Network isolation	`internal: true` on backend	Limits blast radius
No privileged	Never `--privileged`	Maintains container isolation
Pin image versions	`FROM nginx:1.24.0`	Predictable and auditable

Interview Questions — Docker Day 6

Q1. Why is running containers as root dangerous? Answer: Container root (UID 0) maps to host root unless user namespaces are configured. If an attacker exploits a vulnerability in the containerized app they get root inside the container. Combined with container escape vulnerabilities this means root access on the host — full server compromise. Always create and use a dedicated non-root user.

Q2. What is the principle of least privilege in Docker? Answer: Give containers only the minimum permissions needed to function. Drop all Linux capabilities with --cap-drop ALL then add only specific ones needed. Use non-root users. Mount volumes read-only unless writes are needed. Use read-only filesystem with tmpfs only where writes are required.

Q3. How do you prevent secrets from being exposed in Docker images? Answer: Never use ENV for secrets in Dockerfile — visible in docker inspect and docker history. Pass secrets at runtime with -e or --env-file. For production use Docker secrets, AWS Secrets Manager or HashiCorp Vault. In multi-stage builds, secrets used during build never appear in final image.

Q4. What is image vulnerability scanning and why is it important? Answer: Scanning checks image packages against known CVE databases to find security vulnerabilities. Tools like Trivy or Docker Scout identify critical issues before deployment. Base images like ubuntu contain hundreds of packages — many with known exploits. Regular scanning and rebuilding with latest base images is essential.

Q5. Why is mounting the Docker socket dangerous? Answer: /var/run/docker.sock gives full control of the Docker daemon. A container with socket access can create new privileged containers with host filesystem mounts — effectively giving root access to the host. Never mount the Docker socket in production containers.

Q6. What does `--cap-drop ALL --cap-add NET_BIND_SERVICE` do? Answer: Removes all Linux capabilities from the container then adds back only NET_BIND_SERVICE which

allows binding to ports below 1024. This follows least privilege — nginx needs to bind to port 80 but does not need to mount filesystems, load kernel modules or perform other privileged operations.

Q7. What is `no-new-privileges` security option? Answer: Prevents any process inside the container from gaining additional privileges through setuid binaries or sudo. Even if an attacker finds a setuid binary in the container they cannot use it to escalate privileges. Should be set on all production containers.

Q8. How do you use resource limits to improve security? Answer: Memory and CPU limits prevent a compromised container from consuming all host resources — a common technique used by crypto miners after container compromise. Set --memory and --cpus limits on all production containers so one misbehaving container cannot bring down the entire host.

Homework — Before Docker Day 7

```bash
cd ~/linux_practice/docker_practice
```

1. Take your Day 2 Dockerfile and add all security practices:
   - Create non-root user
   - Add `USER` instruction
   - Add `HEALTHCHECK`
   - Pin base image to specific version
2. Run with security flags:

```bash
docker run \
  --read-only \
  --cap-drop ALL \
  --memory 256m \
  --cpus 0.5 \
  --user 1001:1001 \
  --security-opt no-new-privileges \
  myapp
```

3. Install Trivy and scan your image:

```bash
sudo apt install -y trivy
trivy image myapp:latest
```

4. Run `docker history myapp` — verify no secrets visible
5. Write a secure `docker-compose.yml` using the checklist from today

Your Progress

```
Linux   ▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬ ✅ COMPLETE
Git     ▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬ ✅ COMPLETE
Docker  ▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬▬  Day 6 of 8 — 75%
AWS     ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  (parallel chat)
CI/CD   ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  After Docker
K8s     ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  After CI/CD
```

Docker Day 7 we build a **Real DevOps Project** — a complete application with Dockerfile, Docker Compose, security applied, pushed to registry, with a deployment script. Everything from Days 1-6 combined into one real working project. 💪

Say **"Docker Day 7"** whenever you are ready!

Docker Day 7 — Real DevOps Project

What We Are Building Today

Today is different from all previous days. No new theory. Instead we take **everything from Days 1-6 and build one complete real-world project** from scratch.

This is exactly what interviewers mean when they say "show me a project."

The Project — DevOps Dashboard Application

A production-ready three-tier web application:

```
Internet
    ↓
[Nginx — Reverse Proxy + SSL termination]  :80/:443
    ↓
[Flask App — REST API]                      :5000
    ↓
[MySQL Database]                            :3306
    ↓
[Redis Cache]                               :6379

All containers:
- Non-root users
- Resource limits
```

```
- Health checks
- Persistent volumes
- Custom network isolation
- Secrets via env files
- Pushed to Docker Hub
```

Project Structure

```
devops-dashboard/
├── app/
|   ├── Dockerfile
|   ├── requirements.txt
|   └── app.py
├── nginx/
|   ├── Dockerfile
|   └── nginx.conf
├── mysql/
|   └── init.sql
├── scripts/
|   ├── build.sh
|   ├── deploy.sh
|   └── health-check.sh
├── docker-compose.yml
├── docker-compose.override.yml
├── docker-compose.prod.yml
├── .env.example
├── .env
├── .dockerignore
└── README.md
```

Step 1 — Set Up Project

bash

```bash
cd ~/linux_practice
mkdir devops-dashboard
cd devops-dashboard
mkdir app nginx mysql scripts
```

Step 2 — The Flask Application

bash

```bash
vim app/app.py
```

python

```python
from flask import Flask, jsonify
import mysql.connector
import redis
import os
import socket
from datetime import datetime

app = Flask(__name__)

def get_db():
    return mysql.connector.connect(
        host=os.environ.get('DB_HOST', 'database'),
        user=os.environ.get('DB_USER', 'appuser'),
        password=os.environ.get('DB_PASSWORD'),
        database=os.environ.get('DB_NAME', 'devops_db')
    )

def get_redis():
    return redis.Redis(
        host=os.environ.get('REDIS_HOST', 'cache'),
        port=int(os.environ.get('REDIS_PORT', 6379)),
        decode_responses=True
    )

@app.route('/health')
def health():
    return jsonify({
        'status': 'healthy',
        'container': socket.gethostname(),
        'timestamp': datetime.utcnow().isoformat()
    }), 200

@app.route('/api/info')
def info():
    return jsonify({
        'app': 'DevOps Dashboard',
        'version': os.environ.get('APP_VERSION', '1.0.0'),
        'environment': os.environ.get('FLASK_ENV', 'production'),
        'container_id': socket.gethostname()
    })

@app.route('/api/db-status')
def db_status():
    try:
        conn = get_db()
        cursor = conn.cursor()
```

```
        cursor.execute('SELECT VERSION()')
        version = cursor.fetchone()[0]
        conn.close()
        return jsonify({'database': 'connected', 'version': version})
    except Exception as e:
        return jsonify({'database': 'error', 'message': str(e)}), 500

@app.route('/api/cache-status')
def cache_status():
    try:
        r = get_redis()
        r.ping()
        hits = r.get('cache_hits') or 0
        return jsonify({'cache': 'connected', 'hits': hits})
    except Exception as e:
        return jsonify({'cache': 'error', 'message': str(e)}), 500

@app.route('/api/visit')
def visit():
    try:
        r = get_redis()
        visits = r.incr('visit_count')
        return jsonify({'visits': visits})
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step 3 — Requirements File

```bash
vim app/requirements.txt
```

```
flask==3.0.0
mysql-connector-python==8.2.0
redis==5.0.1
gunicorn==21.2.0
```

Step 4 — Application Dockerfile

```bash
vim app/Dockerfile
```

dockerfile

```dockerfile
# ── Stage 1: Build ──────────────────────────────
FROM python:3.11-slim AS builder

WORKDIR /build

# Install build dependencies
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        gcc \
        default-libmysqlclient-dev \
        pkg-config \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# ── Stage 2: Production ─────────────────────────
FROM python:3.11-slim AS production

LABEL maintainer="varun@company.com"
LABEL version="1.0.0"
LABEL description="DevOps Dashboard Flask Application"

# Install only runtime dependencies
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        default-libmysqlclient-dev \
        curl \
    && rm -rf /var/lib/apt/lists/*

# Create non-root user
RUN groupadd -r appgroup && \
    useradd -r -g appgroup -d /app -s /bin/false appuser

WORKDIR /app

# Copy installed packages from builder
COPY --from=builder /root/.local /home/appuser/.local

# Copy application code
COPY --chown=appuser:appgroup app.py .

# Create necessary directories with correct ownership
RUN mkdir -p /app/logs && \
    chown -R appuser:appgroup /app
```

```
# Switch to non-root user
USER appuser

# Set Python path
ENV PATH=/home/appuser/.local/bin:$PATH
ENV PYTHONUNBUFFERED=1
ENV FLASK_APP=app.py

EXPOSE 5000

# Health check
HEALTHCHECK --interval=30s --timeout=5s --start-period=15s --retries=3 \
    CMD curl -f http://localhost:5000/health || exit 1

# Use gunicorn for production
CMD ["gunicorn", \
     "--bind", "0.0.0.0:5000", \
     "--workers", "2", \
     "--timeout", "60", \
     "--access-logfile", "-", \
     "app:app"]
```

Step 5 — Nginx Configuration

```bash
vim nginx/nginx.conf
```

```nginx
upstream flask_app {
    server app:5000;
    keepalive 32;
}

server {
    listen 80;
    server_name localhost;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    add_header X-XSS-Protection "1; mode=block";

    # Logging
    access_log /var/log/nginx/access.log;
```

```
    error_log /var/log/nginx/error.log;

    # Health check endpoint
    location /health {
        access_log off;
        proxy_pass http://flask_app/health;
    }

    # API routes
    location /api/ {
        proxy_pass http://flask_app;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_connect_timeout 30s;
        proxy_read_timeout 60s;
    }

    # All other routes
    location / {
        proxy_pass http://flask_app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

bash

```bash
vim nginx/Dockerfile
```

dockerfile

```dockerfile
FROM nginx:1.24-alpine

LABEL maintainer="varun@company.com"

# Remove default config
RUN rm /etc/nginx/conf.d/default.conf

# Copy custom config
COPY nginx.conf /etc/nginx/conf.d/

# Create non-root nginx user directories
RUN mkdir -p /var/cache/nginx/client_temp \
             /var/cache/nginx/proxy_temp \
             /var/cache/nginx/fastcgi_temp \
```

```
             /var/cache/nginx/uwsgi_temp \
             /var/cache/nginx/scgi_temp \
    && chown -R nginx:nginx /var/cache/nginx \
    && chown -R nginx:nginx /var/log/nginx \
    && touch /var/run/nginx.pid \
    && chown nginx:nginx /var/run/nginx.pid

USER nginx

EXPOSE 80

HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
    CMD curl -f http://localhost/health || exit 1
```

Step 6 — Database Initialization

```bash
vim mysql/init.sql
```

```sql
CREATE DATABASE IF NOT EXISTS devops_db;

USE devops_db;

CREATE TABLE IF NOT EXISTS visits (
    id INT AUTO_INCREMENT PRIMARY KEY,
    container_id VARCHAR(64),
    visited_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ip_address VARCHAR(45)
);

CREATE TABLE IF NOT EXISTS deployments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    version VARCHAR(20),
    deployed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deployed_by VARCHAR(100),
    status ENUM('success', 'failed', 'rollback') DEFAULT 'success'
);

INSERT INTO deployments (version, deployed_by, status)
VALUES ('1.0.0', 'varun', 'success');

GRANT ALL PRIVILEGES ON devops_db.* TO 'appuser'@'%';
FLUSH PRIVILEGES;
```

Step 7 — Environment Variables

```bash
vim .env.example
```

```bash
# Copy this to .env and fill in real values
# NEVER commit .env to Git

# Database
DB_ROOT_PASSWORD=changeme_root
DB_NAME=devops_db
DB_USER=appuser
DB_PASSWORD=changeme_app

# Application
APP_VERSION=1.0.0
FLASK_ENV=production
SECRET_KEY=changeme_secret_key

# Redis
REDIS_HOST=cache
REDIS_PORT=6379
```

```bash
# Create actual .env from example:
cp .env.example .env
vim .env
# Fill in real passwords
```

Step 8 — .dockerignore

```bash
vim .dockerignore
```

```
.git/
.gitignore
.env
*.env
*.log
__pycache__/
*.pyc
```

```
*.pyo
.pytest_cache/
tests/
docs/
*.md
.DS_Store
docker-compose*.yml
scripts/
mysql/
nginx/
```

Step 9 — Main docker-compose.yml

```bash
vim docker-compose.yml
```

```yaml
version: "3.9"

services:

  # ── Database ──────────────────────────────────────────
  database:
    image: mysql:8.0.33
    container_name: dashboard-db
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    volumes:
      - mysql-data:/var/lib/mysql
      - ./mysql/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    networks:
      - backend
    cap_drop:
      - ALL
    cap_add:
      - CHOWN
      - SETUID
      - SETGID
      - DAC_OVERRIDE
    security_opt:
      - no-new-privileges:true
```

```yaml
    deploy:
      resources:
        limits:
          memory: 512M
          cpus: "0.5"
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost",
             "-u", "root", "-p${DB_ROOT_PASSWORD}"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s

  # ── Redis Cache ──────────────────────────────────────────
  cache:
    image: redis:7.2-alpine
    container_name: dashboard-cache
    restart: unless-stopped
    command: redis-server --appendonly yes --requirepass ${REDIS_PASSWORD:-cachepass
    volumes:
      - redis-data:/data
    networks:
      - backend
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
    deploy:
      resources:
        limits:
          memory: 128M
          cpus: "0.25"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 3

  # ── Flask Application ────────────────────────────────────
  app:
    build:
      context: ./app
      dockerfile: Dockerfile
      target: production
    image: ${DOCKER_USERNAME:-varun}/devops-dashboard:${APP_VERSION:-1.0.0}
    container_name: dashboard-app
    restart: unless-stopped
    environment:
      - DB_HOST=database
```

```yaml
    - DB_USER=${DB_USER}
    - DB_PASSWORD=${DB_PASSWORD}
    - DB_NAME=${DB_NAME}
    - REDIS_HOST=cache
    - REDIS_PORT=6379
    - FLASK_ENV=${FLASK_ENV:-production}
    - APP_VERSION=${APP_VERSION:-1.0.0}
  volumes:
    - app-logs:/app/logs
  networks:
    - frontend
    - backend
  cap_drop:
    - ALL
  cap_add:
    - NET_BIND_SERVICE
  security_opt:
    - no-new-privileges:true
  deploy:
    resources:
      limits:
        memory: 256M
        cpus: "0.5"
  depends_on:
    database:
      condition: service_healthy
    cache:
      condition: service_healthy
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
    interval: 30s
    timeout: 5s
    retries: 3
    start_period: 15s

# ── Nginx Proxy ──────────────────────────────────────────
nginx:
  build:
    context: ./nginx
    dockerfile: Dockerfile
  image: ${DOCKER_USERNAME:-varun}/devops-dashboard-nginx:${APP_VERSION:-1.0.0}
  container_name: dashboard-nginx
  restart: unless-stopped
  ports:
    - "80:80"
  volumes:
    - nginx-logs:/var/log/nginx
  networks:
    - frontend
```

```yaml
    cap_drop:
      - ALL
    cap_add:
      - NET_BIND_SERVICE
      - CHOWN
      - SETUID
      - SETGID
    security_opt:
      - no-new-privileges:true
    deploy:
      resources:
        limits:
          memory: 128M
          cpus: "0.25"
    depends_on:
      app:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/health"]
      interval: 30s
      timeout: 3s
      retries: 3

# ── Volumes ──────────────────────────────────────────────
volumes:
  mysql-data:
    driver: local
  redis-data:
    driver: local
  app-logs:
    driver: local
  nginx-logs:
    driver: local

# ── Networks ─────────────────────────────────────────────
networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
    internal: true
```

Step 10 — Development Override

```bash
```

```bash
vim docker-compose.override.yml
```

```yaml
version: "3.9"

services:
  app:
    build:
      target: production
    volumes:
      - ./app:/app                  # live code reload
    environment:
      - FLASK_ENV=development
      - FLASK_DEBUG=1
    ports:
      - "5000:5000"                 # direct access in dev

  database:
    ports:
      - "3306:3306"                 # expose for local DB tools

  cache:
    ports:
      - "6379:6379"                 # expose for local Redis tools
```

Step 11 — Build Script

```bash
vim scripts/build.sh
chmod +x scripts/build.sh
```

```bash
#!/bin/bash
set -euo pipefail

# ── Configuration ──────────────────────────────
DOCKER_USERNAME=${DOCKER_USERNAME:-varun}
APP_VERSION=${APP_VERSION:-1.0.0}
GIT_SHA=$(git rev-parse --short HEAD 2>/dev/null || echo "nogit")
IMAGE_APP="${DOCKER_USERNAME}/devops-dashboard"
IMAGE_NGINX="${DOCKER_USERNAME}/devops-dashboard-nginx"

log() { echo "[$(date '+%H:%M:%S')] $1"; }
```

```bash
# ── Build ──────────────────────────────────────────
log "Building images..."
log "Version: ${APP_VERSION} | SHA: ${GIT_SHA}"

docker compose build --no-cache

# ── Tag ────────────────────────────────────────────
log "Tagging images..."

# App image
docker tag ${IMAGE_APP}:${APP_VERSION} ${IMAGE_APP}:${GIT_SHA}
docker tag ${IMAGE_APP}:${APP_VERSION} ${IMAGE_APP}:latest

# Nginx image
docker tag ${IMAGE_NGINX}:${APP_VERSION} ${IMAGE_NGINX}:${GIT_SHA}
docker tag ${IMAGE_NGINX}:${APP_VERSION} ${IMAGE_NGINX}:latest

log "Images built and tagged successfully"
docker images | grep devops-dashboard
```

Step 12 — Deploy Script

bash

```bash
vim scripts/deploy.sh
chmod +x scripts/deploy.sh
```

bash

```bash
#!/bin/bash
set -euo pipefail

DOCKER_USERNAME=${DOCKER_USERNAME:-varun}
APP_VERSION=${APP_VERSION:-1.0.0}
GIT_SHA=$(git rev-parse --short HEAD 2>/dev/null || echo "nogit")

log()     { echo "[$(date '+%H:%M:%S')] $1"; }
success() { echo "[$(date '+%H:%M:%S')] ✓ $1"; }
error()   { echo "[$(date '+%H:%M:%S')] ✗ $1" >&2; exit 1; }

# ── Pre-flight checks ──────────────────────────────
log "Starting deployment v${APP_VERSION}..."

# Check .env exists
[ -f .env ] || error ".env file not found. Copy .env.example to .env"

# Check Docker is running
```

```
docker info > /dev/null 2>&1 || error "Docker is not running"

# Check disk space
DISK_USAGE=$(df -h / | tail -1 | awk '{print $5}' | tr -d '%')
[ "${DISK_USAGE}" -lt 85 ] || error "Disk usage at ${DISK_USAGE}% — too high to depl

success "Pre-flight checks passed"

# ── Pull latest images ──────────────────────────
log "Pulling latest images..."
docker compose pull --ignore-pull-failures || true

# ── Start services ──────────────────────────────
log "Starting services..."
docker compose up -d --build

# ── Wait for health ─────────────────────────────
log "Waiting for services to be healthy..."
MAX_WAIT=120
WAITED=0

while [ $WAITED -lt $MAX_WAIT ]; do
    UNHEALTHY=$(docker compose ps --format json 2>/dev/null | \
        python3 -c "
import sys, json
services = [json.loads(l) for l in sys.stdin if l.strip()]
unhealthy = [s['Name'] for s in services if s.get('Health','') not in ('healthy','')
print(len(unhealthy))
" 2>/dev/null || echo "0")

    if [ "$UNHEALTHY" = "0" ]; then
        break
    fi

    log "Waiting... (${WAITED}s/${MAX_WAIT}s)"
    sleep 5
    WAITED=$((WAITED + 5))
done

# ── Verify endpoints ────────────────────────────
log "Verifying application..."

HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" \
    http://localhost/health || echo "000")

if [ "$HTTP_CODE" = "200" ]; then
    success "Application is healthy — HTTP ${HTTP_CODE}"
else
    error "Health check failed — HTTP ${HTTP_CODE}"
```

```bash
fi

# ── Cleanup old images ──────────────────────────
log "Cleaning up old images..."
docker image prune -f

success "Deployment complete!"
log "Version: ${APP_VERSION} | SHA: ${GIT_SHA}"
docker compose ps
```

Step 13 — Health Check Script

```bash
vim scripts/health-check.sh
chmod +x scripts/health-check.sh
```

```bash
#!/bin/bash

PASS=0
FAIL=0

check() {
    local name=$1
    local cmd=$2
    if eval "$cmd" > /dev/null 2>&1; then
        echo "  ✓ $name"
        PASS=$((PASS + 1))
    else
        echo "  ✗ $name"
        FAIL=$((FAIL + 1))
    fi
}

echo ""
echo "=== DevOps Dashboard Health Check ==="
echo ""

echo "Container Status:"
check "Nginx running"    "docker compose ps nginx | grep -q running"
check "App running"      "docker compose ps app   | grep -q running"
check "Database running" "docker compose ps database | grep -q running"
check "Cache running"    "docker compose ps cache | grep -q running"

echo ""
```

```bash
echo "Endpoints:"
check "Nginx health"     "curl -sf http://localhost/health"
check "App health"       "curl -sf http://localhost:5000/health"
check "DB status"        "curl -sf http://localhost/api/db-status"
check "Cache status"     "curl -sf http://localhost/api/cache-status"

echo ""
echo "Resources:"
check "Disk < 85%"       "[ $(df -h / | tail -1 | awk '{print $5}' | tr -d '%') -lt
check "Memory available" "[ $(free | awk '/^Mem:/{print int($3/$2*100)}') -lt 90 ]"

echo ""
echo "Result: ${PASS} passed, ${FAIL} failed"
[ $FAIL -eq 0 ] && echo "Status: HEALTHY" || echo "Status: DEGRADED"
echo ""
```

Step 14 — Build and Run

```bash
# Build everything:
bash scripts/build.sh

# Deploy:
bash scripts/deploy.sh

# Check health:
bash scripts/health-check.sh
```

Step 15 — Test Every Endpoint

```bash
# Health check:
curl http://localhost/health

# App info:
curl http://localhost/api/info

# Database status:
curl http://localhost/api/db-status

# Cache status:
curl http://localhost/api/cache-status

# Increment visit counter:
```

```
curl http://localhost/api/visit
curl http://localhost/api/visit
curl http://localhost/api/visit
```

Step 16 — Push to Docker Hub

```bash
# Login to Docker Hub:
docker login

# Push images:
docker push varun/devops-dashboard:1.0.0
docker push varun/devops-dashboard:latest
docker push varun/devops-dashboard-nginx:1.0.0
docker push varun/devops-dashboard-nginx:latest

# Verify on hub.docker.com
```

Step 17 — Useful Day-to-Day Commands

```bash
# View all logs live:
docker compose logs -f

# View only app logs:
docker compose logs -f app

# Check resource usage:
docker stats

# Restart only one service:
docker compose restart app

# Scale app (multiple instances):
docker compose up -d --scale app=3

# Execute shell in running container:
docker compose exec app bash
docker compose exec database mysql -u root -p

# View nginx access logs:
docker compose exec nginx tail -f /var/log/nginx/access.log

# Full teardown:
```

```
docker compose down -v     # WARNING: removes all data

# Partial teardown (keep data):
docker compose down
```

What This Project Demonstrates

When an interviewer asks "show me a project" — this project shows:

Skill	Where it is shown
Multi-stage Dockerfile	`app/Dockerfile`
Non-root containers	`USER appuser` in all Dockerfiles
Health checks	All four services
Volume persistence	`mysql-data`, `redis-data`
Network isolation	`backend` network is `internal: true`
Resource limits	`deploy.resources.limits` on all services
Security options	`cap_drop`, `no-new-privileges` everywhere
Secrets via env	`.env` file pattern
Multi-service Compose	4 services working together
Build automation	`scripts/build.sh`
Deploy automation	`scripts/deploy.sh`
Health monitoring	`scripts/health-check.sh`
Registry push	Docker Hub tags and push
Real application	Actual endpoints that respond

Interview Questions — Docker Day 7

Q1. Walk me through your Docker project. Answer: I built a three-tier application with Flask, MySQL, Redis and Nginx. Each service runs in its own container. The database and cache are on an internal network — not accessible from outside. Only Nginx is exposed on port 80. All containers use non-root users, resource limits and health checks. The application and database data persist using named volumes. I have build and deploy scripts that automate the entire workflow.

Q2. How do you handle zero-downtime deployments with Docker? Answer: The deploy script pulls new images, rebuilds containers and waits for health checks to pass before

marking deployment complete. Using depends_on with service_healthy ensures services start in correct order. For true zero-downtime with multiple replicas you use Docker Swarm or Kubernetes with rolling updates.

Q3. How do you debug a container that keeps crashing? Answer: First check `docker compose logs servicename` for error messages. Then `docker compose ps` to see restart count. Try `docker run -it --entrypoint bash imagename` to get a shell and investigate manually. Check `docker inspect containername` for configuration issues. Use `docker stats` to see if it is resource-constrained.

Q4. How do you ensure the database is ready before the app starts? Answer: Use `depends_on` with `condition: service_healthy` combined with a healthcheck on the database service. The healthcheck runs `mysqladmin ping` every 10 seconds. The app container only starts once the database reports healthy — not just when the container starts.

Q5. How would you handle configuration differences between dev and production? Answer: Base configuration in `docker-compose.yml`. Development overrides in `docker-compose.override.yml` — automatically merged in development with bind mounts for live reload and exposed ports for debugging. Production uses `docker-compose.prod.yml` with `docker compose -f docker-compose.yml -f docker-compose.prod.yml up`.

Homework — Before Docker Day 8

1. Build and run the complete project
2. Test all endpoints with curl
3. Stop the database container — watch app handle errors gracefully
4. Remove all containers but keep volumes — restart and verify data persists
5. Push both images to Docker Hub
6. Run `bash scripts/health-check.sh` and read the output
7. Add one more endpoint to `app.py` — rebuild and redeploy using `scripts/deploy.sh`

Your Progress

```
Linux   ✅ COMPLETE
Git     ✅ COMPLETE
Docker   Day 7 of 8 — 87%
AWS      (parallel chat)
CI/CD    After Docker
K8s      After CI/CD
```

One more day — **Docker Day 8 is the Interview Mega Revision** — every Docker concept reviewed in interview format, scenario questions, common traps and the questions that actually get asked at 10-12 LPA companies. 💪

Say **"Docker Day 8"** whenever you are ready!

Docker Day 8 — Interview Mega Revision

This is Your Final Docker Day 🎉

You have covered everything. Today we go through every concept in interview format — exactly how a 10-12 LPA DevOps interview tests Docker knowledge.

How Docker is Tested at 10-12 LPA Level

```
Round 2 — Technical Interview
├── Conceptual questions     30%  "What is X, difference between X and Y"
├── Command questions        25%  "How do you do X"
├── Scenario questions       30%  "Production is down, what do you do"
└── Show your project        15%  "Walk me through something you built"
```

SECTION 1 — Core Concepts

Q1. What is Docker and what problem does it solve?

Answer: Docker is a containerization platform that packages applications with all their dependencies — code, runtime, libraries, configuration — into a portable container. It solves the "works on my machine" problem.

Before Docker: developer builds on laptop with Python 3.9, server runs Python 3.6 — application crashes. With Docker: the container carries its own Python version. Runs identically on every machine.

Additional problems it solves:

- Consistent environments across dev, staging, production
- Fast deployment — containers start in seconds not minutes
- Efficient resource usage — multiple containers on one server
- Easy scaling — run 10 containers from one image instantly

Q2. What is the difference between a container and a virtual machine?

Answer:

```
Virtual Machine:                Container:
Full Guest OS per VM            Shares host OS kernel
Several GB each                 Tens of MB each
Minutes to start                Seconds to start
Strong isolation                Process-level isolation
More overhead                   Near-native performance
```

VMs virtualise hardware — each has its own OS kernel. Containers virtualise at OS level — share the host kernel, isolate only processes and filesystems. Containers are lighter and faster. VMs provide stronger isolation.

When to use VMs: When you need different OS, stronger security isolation, or legacy applications. **When to use containers:** When you need fast deployment, consistency and efficiency.

Q3. Explain Docker architecture.

Answer:

```
Docker Client  ——commands——>  Docker Daemon  ——manages——>  Containers
(docker CLI)                  (dockerd)                     Images
                                   |                         Volumes
                                   |                         Networks
                              Docker Registry
                              (Docker Hub/ECR)
```

- **Docker Client** — the CLI you type commands into
- **Docker Daemon (dockerd)** — background service that does the actual work
- **Docker Registry** — stores images (Docker Hub, ECR, private)
- **containerd** — low-level container runtime that daemon uses
- **runc** — actual container creation at OS level

Q4. What happens step by step when you run `docker run nginx`?

Answer:

1. Docker client sends request to Docker daemon
2. Daemon checks if `nginx:latest` image exists locally
3. Not found — daemon contacts Docker Hub
4. Downloads image layers (only layers not already cached)
5. Daemon creates a writable container layer on top of image layers
6. Sets up networking — assigns IP, creates virtual network interface
7. Starts the container process (nginx)

8. Streams output back to terminal

Q5. What are Docker image layers and why do they matter?

Answer: Each Dockerfile instruction creates a read-only layer. Layers are stacked — each adds or modifies the filesystem. They are cached and shared between images.

```
Layer 4: COPY app.py           ← your code
Layer 3: RUN pip install       ← dependencies
Layer 2: RUN apt-get install   ← system packages
Layer 1: FROM python:3.11      ← base OS + Python
```

Why they matter:

- **Caching** — unchanged layers rebuild instantly
- **Sharing** — two images using same base share those layers on disk
- **Efficiency** — only changed layers are pushed/pulled
- **Immutability** — image layers never change — only containers have writable layer

---

SECTION 2 — Dockerfile

Q6. What is the difference between CMD and ENTRYPOINT?

Answer:

```dockerfile
# CMD — default command — completely overridden by docker run arguments
CMD ["python3", "app.py"]
docker run myimage              # runs: python3 app.py
docker run myimage bash         # runs: bash (CMD replaced)

# ENTRYPOINT — fixed executable — arguments appended
ENTRYPOINT ["python3"]
CMD ["app.py"]
docker run myimage              # runs: python3 app.py
docker run myimage test.py      # runs: python3 test.py (CMD replaced, ENTRYPOINT ke
```

Best practice — combine both. ENTRYPOINT for the executable, CMD for default arguments.

Q7. What is a multi-stage build and why use it?

Answer: Multiple FROM statements in one Dockerfile. Each stage builds on previous. Only the final stage becomes the image.

```dockerfile
FROM node:18 AS builder        # stage 1 — has all build tools
RUN npm install && npm run build

FROM node:18-alpine            # stage 2 — only runtime
COPY --from=builder /app/dist ./dist
CMD ["node", "dist/app.js"]
```

Result: 150MB instead of 900MB. Build tools, source code and dev dependencies never enter production image. Smaller image = faster pulls, smaller attack surface, less storage cost.

Q8. Why should you clean up in the same RUN command?

Answer:

```dockerfile
# WRONG — layer created with files, then deleted in separate layer:
RUN apt-get update && apt-get install -y nginx   # layer: 100MB
RUN rm -rf /var/lib/apt/lists/*                  # layer: +0MB, files still in previ

# CORRECT — never exists in any layer:
RUN apt-get update && \
    apt-get install -y nginx && \
    rm -rf /var/lib/apt/lists/*                  # single layer: 40MB
```

Docker images are the sum of all layers. Deleting in a separate RUN creates a new layer that hides the files but does not remove them from the image.

Q9. What is the difference between COPY and ADD?

Answer:

```dockerfile
COPY app.py /app/              # simple file copy — explicit and predictable
ADD app.tar.gz /app/           # copies AND auto-extracts archives
ADD https://example.com/f /app/ # can download from URLs
```

Use `COPY` for simple copying — it is explicit and has no surprises. Use `ADD` only when you specifically need automatic tar extraction. Never use ADD for URLs — use curl/wget in RUN instead (better caching, clearer).

Q10. What is the purpose of .dockerignore?

Answer: Excludes files from the build context sent to Docker daemon. Without it large directories like `node_modules` (500MB) or `.git` are sent unnecessarily — slowing builds and potentially including secrets.

Always exclude:

- `node_modules/`, `vendor/`, `__pycache__/` — reinstalled during build
- `.git/` — not needed in image
- `.env` — secrets must never be in image
- `*.log` — not needed
- Test files — not needed in production

SECTION 3 — Volumes and Networking

Q11. What is the difference between a named volume and a bind mount?

Answer:

```bash
# Named volume — Docker manages storage:
-v mysql-data:/var/lib/mysql
# Lives at: /var/lib/docker/volumes/mysql-data/_data
# Best for: production data, databases

# Bind mount — you specify exact host path:
-v $(pwd)/html:/usr/share/nginx/html
# Lives at: your specified path
# Best for: development, live code reload
```

Named volumes are portable — work on any machine. Bind mounts depend on exact host path existing. Named volumes survive `docker compose down`, bind mounts reflect live host changes.

Q12. What happens to data when a container is removed?

Answer: Data written inside the container's writable layer is permanently lost when the container is removed. Data in named volumes or bind mounts is NOT affected — it exists independently of the container lifecycle.

This is why databases must always use named volumes. `docker compose down` removes containers. `docker compose down -v` removes containers AND volumes — destroys all

data.

Q13. What is the difference between the default bridge network and a custom bridge network?

Answer:

```bash
# Default bridge:
docker run --name app1 nginx
docker run --name app2 nginx
# app2 CANNOT reach app1 by name — only by IP address

# Custom bridge:
docker network create mynet
docker run --name app1 --network mynet nginx
docker run --name app2 --network mynet nginx
# app2 CAN reach app1 using hostname "app1" — automatic DNS
```

Custom networks provide automatic DNS resolution — containers find each other by name. Default bridge only allows IP-based communication. Always use custom networks for multi-container applications.

Q14. What does `internal: true` do on a Docker network?

Answer: Prevents containers on that network from accessing the internet or external networks. They can only communicate with other containers on the same network.

```yaml
networks:
  backend:
    internal: true    # database cannot make outbound internet requests
```

Used for backend networks containing databases — they have no reason to reach the internet. If a database container is compromised, `internal: true` limits the damage.

SECTION 4 — Docker Compose

Q15. What is Docker Compose and why use it?

Answer: A tool for defining and running multi-container applications using a YAML file. Replaces multiple `docker run` commands with a single file and single command.

Benefits:

- **Reproducible** — same file produces identical environment every time
- **Version controlled** — `docker-compose.yml` lives in Git
- **Simple** — `docker compose up -d` starts everything
- **Network automatic** — creates network and connects services automatically
- **Volume automatic** — creates named volumes automatically

Q16. What is the limitation of `depends_on` and how do you fix it?

Answer:

```yaml
# WRONG — depends_on only waits for container to START:
depends_on:
  - database
# Database container starts but MySQL takes 20 seconds to initialize
# App starts immediately, tries to connect, fails

# CORRECT — wait for service_healthy:
depends_on:
  database:
    condition: service_healthy

# Combined with healthcheck on database:
database:
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
    interval: 10s
    retries: 5
```

`condition: service_healthy` waits until the healthcheck passes — not just until the container starts. This is one of the most common Docker Compose mistakes in production.

Q17. What is the difference between docker compose down and docker compose stop?

Answer:

```bash
docker compose stop          # stops containers — keeps containers and networks
docker compose start         # can restart them quickly

docker compose down          # stops AND removes containers and networks
docker compose down -v       # also removes volumes — DELETES ALL DATA permanently
```

Use `stop` / `start` for temporary pausing — development workflow. Use `down` for clean environment reset. Never use `down -v` on production without a backup.

Q18. How do you use different configs for dev and production in Compose?

Answer:

```bash
# docker-compose.yml — base config
# docker-compose.override.yml — auto-merged in development
# docker-compose.prod.yml — explicit production config

# Development (automatic merge):
docker compose up -d
# Uses: docker-compose.yml + docker-compose.override.yml

# Production (explicit):
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
# Uses: only the files you specify
```

Override file adds dev-specific things — bind mounts for live reload, exposed database ports, debug environment variables.

SECTION 5 — Security

Q19. What are the top 5 Docker security best practices?

Answer:

1. **Non-root user** — `USER appuser` in Dockerfile. Container root = host root risk.
2. **Read-only filesystem** — `--read-only` prevents writing backdoors.
3. **No secrets in image** — pass at runtime with `-e` or `--env-file`. Never in ENV instruction.
4. **Drop capabilities** — `--cap-drop ALL --cap-add only-needed`. Least privilege.
5. **Resource limits** — `--memory` and `--cpus` prevent DoS and crypto mining.

Bonus: Scan images with Trivy, use minimal base images, never mount Docker socket, pin image versions.

Q20. How does a container escape attack work?

Answer: An attacker:

1. Finds vulnerability in your containerized application
2. Executes code inside container
3. If container runs as root — they have root inside container
4. Exploit kernel vulnerability to break container namespace isolation
5. Now have root on host machine

Prevention:

- Non-root user eliminates step 3 risk
- `--cap-drop ALL` removes capabilities needed for many exploits
- `no-new-privileges` prevents privilege escalation inside container
- Updated base images patch known kernel vulnerabilities
- `seccomp` profiles limit allowed system calls

Q21. What is the danger of mounting /var/run/docker.sock?

Answer: The Docker socket gives full control of the Docker daemon. A container with socket access can:

```bash
# Inside compromised container with socket mounted:
docker run -it \
  --privileged \
  -v /:/hostroot \
  ubuntu bash
# Now have read/write access to entire host filesystem as root
```

This is called "container breakout via Docker socket." Never mount the socket in production. Use Docker-in-Docker (`docker:dind`) for CI/CD that needs Docker access.

SECTION 6 — Registry and Production

Q22. Why should you never use `latest` tag in production?

Answer: `latest` is just a tag — it does not automatically update. Two servers pulling `latest` at different times may get completely different images. You cannot tell what version is actually running.

Best practice:

```bash
```

```bash
# Development: latest is acceptable
# Staging: semantic version — v1.2.3
# Production: git SHA + semantic version
docker tag myapp registry/myapp:1.2.3
docker tag myapp registry/myapp:a1b2c3d    # git SHA — fully traceable
```

If production breaks — `docker inspect container` shows the SHA, you know exactly which commit is running.

Q23. What is the difference between Docker Hub and AWS ECR?

Answer:

Feature	Docker Hub	AWS ECR
Type	Public/private cloud	Private cloud
Authentication	Username/password or token	AWS IAM roles
Integration	Generic	Native AWS — ECS, EKS, CodePipeline
Vulnerability scanning	Basic	Advanced with Inspector
Cost	Free public, paid private	Pay per GB stored
Use case	Open source, personal	Production AWS workloads

ECR preferred for AWS deployments — no separate credentials, uses existing IAM, faster pulls within AWS.

SECTION 7 — Scenario Questions

Q24. Your application container keeps restarting. How do you debug it?

Answer:

```bash
# Step 1 — see restart count and last status:
docker compose ps

# Step 2 — check logs — what error is it printing before crash:
docker compose logs app
docker compose logs --tail 50 app

# Step 3 — check if it even starts:
```

```bash
docker run -it --entrypoint bash myapp:1.0    # override entrypoint to get shell
# Look around manually — are files in right place?

# Step 4 — check if dependencies are healthy:
docker compose ps database
docker compose logs database

# Step 5 — check resource limits:
docker stats
docker inspect myapp | grep -i memory

# Step 6 — check environment variables:
docker inspect myapp | grep -A 20 '"Env"'

# Step 7 — reproduce locally:
docker run -it --env-file .env myapp:1.0 bash
# Try running the app manually inside
```

Q25. Production database is running out of disk space. What do you do?

Answer:

```bash
# Step 1 — confirm the issue:
docker exec database df -h /var/lib/mysql

# Step 2 — check Docker disk usage:
docker system df

# Step 3 — clean Docker system (safely):
docker system prune -f          # removes stopped containers, unused images
docker image prune -a -f        # removes all unused images

# Step 4 — check volume size:
docker volume inspect mysql-data
du -sh /var/lib/docker/volumes/mysql-data/

# Step 5 — check database logs for large tables:
docker exec database mysql -u root -p \
  -e "SELECT table_schema, table_name, \
      ROUND(data_length/1024/1024,2) AS 'MB' \
      FROM information_schema.tables \
      ORDER BY data_length DESC LIMIT 10;"

# Step 6 — if logs are the issue:
docker exec database mysql -u root -p -e "PURGE BINARY LOGS BEFORE NOW();"
```

```
# Long term — set up volume monitoring and alerts
```

Q26. You need to update a production application with zero downtime. How?

Answer:

```bash
# With Docker Compose (simple approach):
# 1. Build new image:
docker build -t myapp:1.2.0 .

# 2. Push to registry:
docker push registry/myapp:1.2.0

# 3. Update .env with new version:
APP_VERSION=1.2.0

# 4. Pull and recreate only app service:
docker compose pull app
docker compose up -d --no-deps app
# --no-deps = don't restart dependencies (database stays up)

# 5. Verify health:
curl http://localhost/health

# 6. If failed — rollback instantly:
APP_VERSION=1.1.0 docker compose up -d --no-deps app
```

For true zero-downtime with multiple replicas — use Docker Swarm rolling updates or Kubernetes.

Q27. A developer says "the app works locally but fails in the Docker container." How do you debug?

Answer:

```bash
# Step 1 — get inside the container exactly as it runs:
docker run -it --env-file .env myapp:1.0 bash

# Step 2 — check environment variables are correct:
env | grep -i db
env | grep -i api
```

```bash
# Step 3 — check file permissions:
ls -la /app/
whoami         # are you running as expected user?

# Step 4 — check network — can container reach services:
curl http://database:3306    # using container name
ping database

# Step 5 — check mounted volumes:
ls -la /app/uploads/         # is volume mounted?
ls -la /run/secrets/         # are secrets available?

# Step 6 — compare with local setup:
# What Python/Node version locally vs in container?
python3 --version
# What packages installed?
pip list

# Step 7 — check base image differences:
docker run -it python:3.11-slim bash
# Compare to local Python environment
```

Q28. How do you monitor Docker containers in production?

Answer:

```bash
# Built-in monitoring:
docker stats                                # live CPU/memory/network
docker events                              # real-time events stream
docker inspect container_name              # detailed config and state

# Health check status:
docker inspect --format='{{.State.Health.Status}}' container_name

# Log monitoring:
docker compose logs -f --tail 100          # follow all logs

# Production monitoring stack:
# Prometheus + cAdvisor — scrapes container metrics
# Grafana — visualizes metrics dashboards
# AlertManager — sends alerts when thresholds exceeded
# ELK Stack — centralised log management

# Quick alert script (from Day 8 Linux):
for container in nginx app database cache; do
    STATUS=$(docker inspect --format='{{.State.Status}}' $container 2>/dev/null)
```

```bash
    if [ "$STATUS" != "running" ]; then
        echo "ALERT: $container is $STATUS"
    fi
done
```

SECTION 8 — Quick Fire Round

Know These Instantly

```bash
# Q: How do you see all containers including stopped?
docker ps -a

# Q: How do you get a shell inside a running container?
docker exec -it container_name bash

# Q: How do you see container logs in real time?
docker logs -f container_name

# Q: How do you remove all stopped containers?
docker container prune

# Q: How do you remove all unused images?
docker image prune -a

# Q: How do you see disk usage by Docker?
docker system df

# Q: How do you run a one-off command in a Compose service?
docker compose run --rm service_name command

# Q: How do you rebuild without cache?
docker build --no-cache -t myapp .
docker compose build --no-cache

# Q: How do you scale a service in Compose?
docker compose up -d --scale web=3

# Q: How do you update one service without restarting others?
docker compose up -d --no-deps --build app

# Q: How do you copy file from container to host?
docker cp container_name:/app/file.txt ./file.txt

# Q: How do you copy file from host to container?
```

```
docker cp ./file.txt container_name:/app/file.txt

# Q: What is a dangling image?
# An image with no tag — created when you rebuild with same tag
# Remove with: docker image prune

# Q: What is the difference between EXPOSE and -p?
# EXPOSE documents which port app uses — informational only
# -p actually maps host port to container port — makes it accessible

# Q: What does --rm flag do?
# Automatically removes container when it stops
# Useful for one-off commands: docker run --rm myapp python migrate.py

# Q: What is docker commit?
docker commit container_name new_image:tag
# Creates image from running container — NOT recommended
# Use Dockerfile instead — reproducible and version-controlled

# Q: What is docker save and docker load?
docker save myapp:1.0 > myapp.tar         # export image to file
docker load < myapp.tar                   # import image from file
# Used for air-gapped environments without registry access
```

SECTION 9 — Complete Docker Summary

Everything You Have Mastered

```
Day 1 ✅  Containers vs VMs, images, docker run, ps, logs, exec
Day 2 ✅  Dockerfile, FROM RUN COPY WORKDIR ENV EXPOSE CMD ENTRYPOINT
          Layer caching, multi-stage builds, .dockerignore
Day 3 ✅  Named volumes, bind mounts, bridge/host/none networks
          Custom networks, container-to-container DNS
Day 4 ✅  Docker Compose, services/volumes/networks in YAML
          depends_on, healthchecks, dev vs prod configs
Day 5 ✅  Docker Hub, ECR, image tagging strategy, push/pull
          Registry in CI/CD pipeline, git SHA tagging
Day 6 ✅  Non-root users, read-only filesystem, secrets management
          Capabilities, resource limits, image scanning, socket security
Day 7 ✅  Real project — Flask + MySQL + Redis + Nginx
          Build scripts, deploy scripts, health monitoring
Day 8 ✅  Interview mega revision — all scenarios covered
```

Phrases That Impress Docker Interviewers

Situation	Say this
Asked about images	"I always use multi-stage builds to keep production images small"
Asked about security	"All our containers run as non-root with capabilities dropped"
Asked about data	"Databases always use named volumes — data persists independently"
Asked about networking	"Backend services use internal networks — no internet access"
Asked about production	"We tag images with git SHA for full traceability"
Asked about monitoring	"Health checks on every service, Prometheus for metrics"
Asked about secrets	"Never in Dockerfile or image — passed at runtime via env files"
Asked about debugging	"First thing I check is docker logs, then docker exec to inspect"

Your Complete Docker Roadmap — Done

```
Linux   ━━━━━━━━━━━━━━━━━━━━━━━━  ✅ COMPLETE — 16 days
Git     ━━━━━━━━━━━━━━━━━━━━━━━━  ✅ COMPLETE — 4 days
Docker  ━━━━━━━━━━━━━━━━━━━━━━━━  ✅ COMPLETE — 8 days
AWS     ┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈  (parallel chat — keep going)
CI/CD   ┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈  NEXT HERE
K8s     ┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈┈  After CI/CD
```

What is Coming Next — CI/CD

CI/CD is where Linux + Git + Docker all come together into a pipeline:

```
You push code to GitHub
          ↓
GitHub Actions triggers automatically
          ↓
Runs your Linux shell scripts
          ↓
Builds Docker image
          ↓
Pushes to ECR registry
          ↓
Deploys to server
          ↓
Zero human intervention
```

Everything you learned becomes one automated pipeline. This is the heart of DevOps.

Say **"make docker document"** to get the complete Docker notes packed into a document, or say **"CI/CD Day 1"** to start the next module! 💪